JHQ-GPU: drafting handbook

Sections 3, 4 and 6 — skeleton, and every figure with its evidence

branch `fix/recall-eval-v15` — generated 2026-09-10 — 16 figures

How to use this. Part I is the drafting skeleton with each figure placed at its section, so the prose and the evidence sit together. Part II is the figure reference: every figure at full width followed by its script's docstring, which is where the measured numbers, the excluded rows and the noise floors are written down. **Write captions from Part II, not from Part I** — the one-line descriptions in Part I are labels, not claims.

Numbers move. `paper_adc2026/README.md` is the evidence map and is authoritative; the "Superseded measurements" section of the skeleton lists what has already been withdrawn. Regenerate with `python3 build_handbook.py` after `figures/make_all.sh`.

Contents

ADC 2026: mother skeleton v3 for Sections 3, 4, and 6

This is the new drafting framework, not a replacement for any earlier skeleton or review. Repository branch: `fix/recall-eval-v15`. Evidence audit: 2026-09-10. Paths below are relative to the repository root; `data/` in this document means `paper_adc2026/data/`. Version identifiers appear only in internal evidence mapping, never in manuscript prose. Existing reviews are advice; frozen results and implementation semantics take precedence. `[TBD]` means the claim or required validation is not ready for publication.

Superseded measurements — read before drafting

Three numbers that appear in earlier skeletons and reviews are older than the evidence. They were true of the runs they came from and are not true of the paper's current data. `README.md` is the evidence map and is current.

older claim	what is measured now	source
"sample size around `S=32`" / "S=32 is the knee"	**The knee is per workload.** 2000 resampled draws scored on held-out queries: at S=32, 76% inside 1e-3 on openai3-3072 and **27% on vogue-768**, where the mean held-out loss is 0.0033 and the 95th percentile 0.0110. openai3-3072 reaches 98% at S=64; vogue-768 still loses 11% of draws at S=128. Give S against a stated risk, not as a constant.	`data/alpha_resample.json`, `figures/alpha_resample.py`
"factorised LUT: roughly +6% to +14.5%"	**-4% at M=96 to +53% at M=384.** All 28 original cells were M=96 and M=128. The 256-entry table is 98 KiB at M=96, which shared memory holds, and 393 KiB at M=384, which it does not: the factorisation matters exactly where the full table stops fitting. One M=96 cell is negative.	`data/lut_groups.log`, `figures/fig_lutgroups.py`
any statement that the coarse quantiser is undertrained	**It is not, in the reported runs.** `_pa.sh` passes `JHQ_N_TRAIN = 39 × nlist` on every dataset. The undertraining finding is historical.	`SKELETON_REVIEW_v2.md`

Two results the skeleton could not have known to ask for, both now measured:

- **Why halves and not quarters.** G=2 beats G=1, G=4 and G=8 in all eight

configurations, and G=4 and G=8 have *smaller* tables yet lose in proportion to their loads a candidate a subspace. The scan is issue-bound — the same conclusion as the table-free distance and the packed load, from a third direction. (§6.4.2, `fig_lutgroups`.)

- **The two payoffs multiply.** The factorisation is worth about the same with

and without the packed layout, so "pays twice" holds as an independent product rather than an overlap — which the framing had been assuming without evidence. (`fig_lutgroups(b)`.)

Paper contract and contribution hierarchy

Keep the teacher-required order: **1 Introduction; 2 Preliminaries; 3 Method Part 1; 4 Method Part 2; 5 Related Work; 6 Experiments; 7 Conclusion.** Detailed literature discussion stays in Section 5.

Paper story:

> We exploit JHQ's Cartesian primary representation to redesign its primary-distance path for GPU execution, and use output-stability calibration to decide how much residual refinement a workload actually needs.

Section 3 asks what JHQ-specific structure is worth exploiting on GPU. Section 4 asks how much refinement to use and when calibration pays back. Section 6 asks whether matched, reproducible experiments support those answers. This is representation, execution, policy, and evidence—not a chronology of CUDA changes.

Component	Scientific status	Required treatment
Cartesian-factorized primary LUT	Strongest JHQ-specific contribution	Derive the exact identity and measure its execution benefit.
Packed primary-code loading	Representation-enabled optimization	Explain admissibility and instruction reduction; secondary to the algebraic contribution.
Subspace-major physical layout	Necessary, standard GPU engineering	Explain coalescing without claiming the transpose as novelty.
Batched GEMM / orthogonal transform	Enabling infrastructure	Preserve mathematical semantics.
Query-sized launch, cursor traversal removal, bug fixes	Implementation corrections	Separate from research contributions and from algebraic ablations.
Output-stability alpha selection	Policy/algorithm contribution	Validate selection and economics separately.

Novelty and effect size are different axes: packing may yield a larger measured speedup than factorization. Report both honestly rather than changing their novelty labels to follow performance. Do not assert that no other quantizer can factorize; the exact factorization follows from this Cartesian representation and is not available to the compared baselines in their current representations. Baseline-specific representation citations remain `[TBD]` for the literature pass.

3 GPU-Native JHQ

3.1 Design Overview

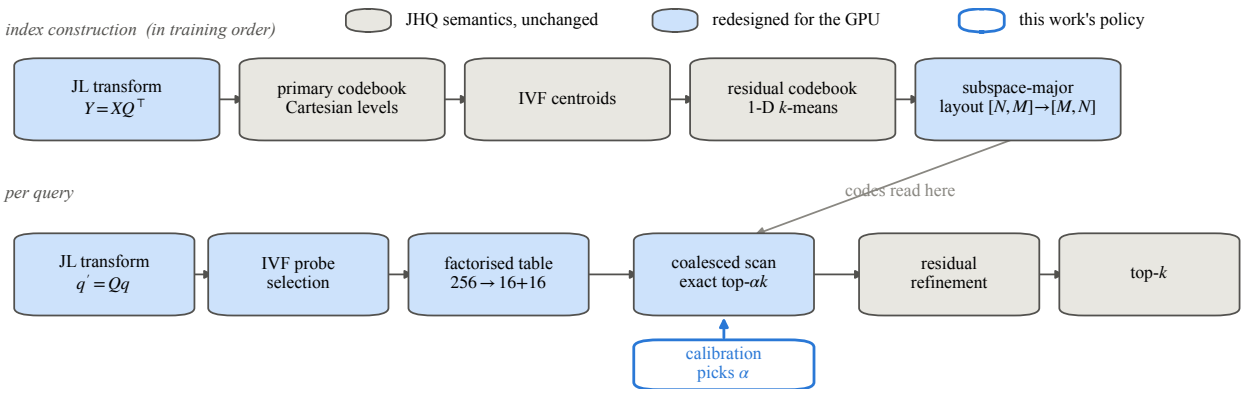


Figure 1: **fig_pipeline.** the JHQ-GPU pipeline, shaded by what changed *Full caption material and every caveat: Part II,* `fig_pipeline.py`.

Introduce the two-level representation and the GPU query path in `fig_pipeline`. A primary scan scores routed candidates, retains primary survivors, and refines only those survivors using residual information before final top-k selection. The representation determines which distance computations can be reorganized exactly; the refinement budget determines how much approximate filtering occurs.

Use consistent notation: database vector x, transformed vector y = Qx, transformed query q' = Qq, Q in R^(d x d), M subspaces, primary code c_m, requested output size k. The JL/orthogonal transform is square, not dimensionality reduction. For row batches, Y = X Q^T is the same convention.

3.2 GPU-Oriented Representation and Layout

Explain batched transformation, primary and residual encoding, IVF organization, and physical layout in a connected account. The residual is **r = y - y_hat_primary**, never the IVF-centroid residual. IVF organizes candidate routing; it does not redefine the residual target. Query ADC uses tables built from the full query, not a single quantized database-style query code.

Describe the logical candidate-major primary matrix [N,M] and the subspace-major scan layout [M,N], followed by the packed physical variant. Keep logical layout and packed storage distinct. Batched GEMM and the transpose are implementation infrastructure, not independent research claims.

Training dependencies in `fig_pipeline` must match code: transformation and primary construction precede IVF centroid training and residual codebook training; the residual codebook uses a sample and need not wait for all database

assignments. Avoid hard-coded equation numbers or a schematic that implies a false dependency.

3.3 Structure-Aware Query Processing

3.3.1 Cartesian-Factorized Primary Distance Table For each subspace, T_m[c] = ||q'_m - C_m[c]||^2. In the current Cartesian 8-bit representation, split the coordinates and code into independent high and low halves. Squared Euclidean distance adds over disjoint coordinates, giving exactly:

T_m[c] = T_m_hi[c » 4] + T_m_lo[c mod 16].

Thus 256 entries become 16 + 16 = 32 entries per subspace. These counts are algebraic consequences, not measured speedups. Explain the chain from Cartesian structure to exact factorization to smaller query-specific tables and less construction work, and then evaluate GPU throughput. Extra lookups and arithmetic mean table-size reduction is not a proportional throughput promise.

`fig_lut` belongs here. The implementation checks Cartesian separability against the centroids; a generic refined codebook cannot be substituted silently. Algebraic exactness does not imply bitwise-identical floating-point evaluation or deterministic tie order.

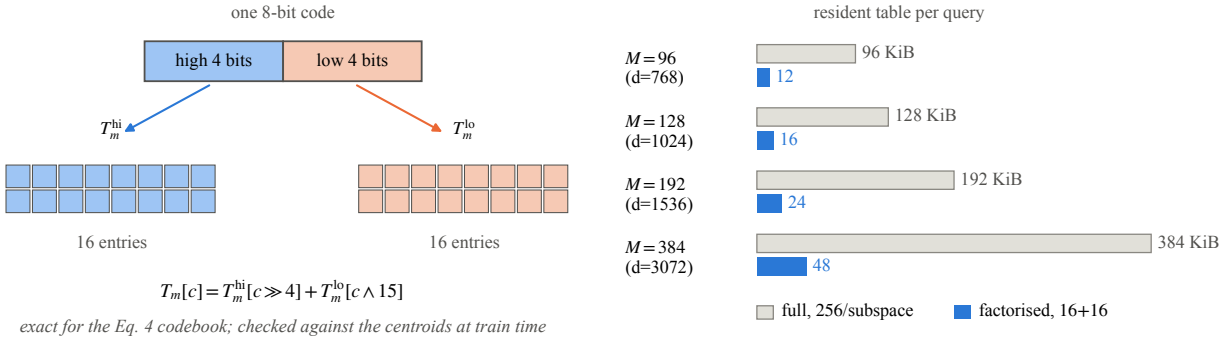


Figure 2: **fig_lut.** the Cartesian factorisation, and what it saves per query *Full caption material and every caveat: Part II,* `fig_lut.py`.

3.3.2 Coalesced Primary Scan and Packed Access Explain contiguous candidate accesses at a fixed subspace, primary distance accumulation, and survivor selection. The current one-byte primary code permits packing four subspaces into one 32-bit word. Present packing as enabled by this representation and as reducing load/address/loop instructions. Do not imply packing necessarily reduces transferred bytes when the byte layout is already coalesced. Do not repeat the incorrect 128-byte-line / 75%-waste explanation.

`fig_layout` separates the coalescing effect of layout from the instruction effect of packing. Query-sized launch and redundant cursor removal belong in implementation notes and a separately labeled correction ablation.

3.3.3 Selective Residual Refinement Explain residual decoding/scoring of primary survivors and final top-k selection. Distinguish exact evaluation of a chosen approximate representation from exact nearest-neighbor search over original vectors: routing and primary filtering can still exclude true neighbors. Motivate the hierarchy using the primary-only ablation in Section 6.4, without implying that exhaustive routing fixes inadequate primary resolution.

3.4 The Remaining Policy Question

The execution path accepts a survivor budget, but does not determine its useful size. End explicitly with **ck = alpha * k** and the question:

> How many primary survivors should actually be refined?

4 Adaptive Hierarchical Refinement

4.1 Motivation and Workload Dependence

Original JHQ leaves alpha externally specified. Too little refinement can lose neighbors; too much can return the same top-k while spending unnecessary residual work.

Use this bounded empirical statement: **The sufficient refinement budget ranges from 4 to at least 200 at nprobe=128; the upper endpoint is a lower bound because the measured curve is still improving at the largest tested alpha.** Evidence: `data/alpha6.log`, Br=8 rows, not `alpha_ds.log`, which does not test 200. OpenAI-3072 recall is 0.9416 at both alpha=4 and 200; arxiv improves from 0.9760 at 100 to 0.9771 at 200. This is an operational, grid-limited saturation observation, not a precise ratio or a proof of a plateau beyond the grid. No "25x" saturation claim.

4.2 Output-Stability Criterion

Let Q_s be S sampled queries and R_max(q) the top-k ID set at a generous alpha_max. Define R_alpha(q) analogously and define **epsilon as a non-negative integer count of disagreeing slots over the whole sample**:

D(alpha) = sum over q in Q_s of [k - |R_alpha(q) intersect R_max(q)|] alpha* = min {alpha in tested grid : D(alpha) <= epsilon}.

This is set overlap per query: returned neighbors in a different order agree. It is neither a fractional tolerance nor epsilon per query nor positional rank disagreement. It assumes valid distinct top-k IDs. Current defaults are S=32 and epsilon=1; these are empirical settings, not a correctness guarantee.

The rule uses no external ground-truth labels and no learned predictor. Ground truth is used only afterward to evaluate the policy. Agreement with alpha_max cannot certify exact-neighbor recall, recover IVF routing misses, or detect improvements beyond alpha_max.

4.3 Selection Procedure and Positioning

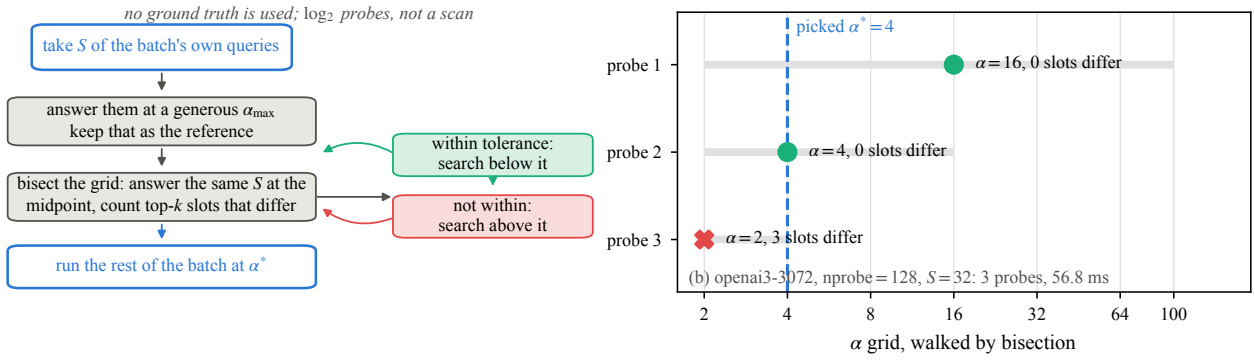


Figure 3: **fig_rule.** how the budget is chosen, and one real calibration *Full caption material and every caveat: Part II, `fig_rule.py`.*

Algorithm box: choose a sample and descending alpha grid; compute the alpha_max reference; evaluate smaller budgets against D; select an accepted budget; apply it to subsequent workload queries. Report the actual search schedule, sample selection, and stopping conditions alongside `fig_rule`.

Internal implementation evidence: `examples/demo_jhq_alpha_fast.cu` defaults to grid {100,64,32,16,8,4,2}, selects evenly strided queries from the current batch, and defaults to bisection. Its optional linear mode descends and stops at first rejection. The conceptual minimum above is the target; claiming the implementation always finds the global minimum requires the accepted-prefix/monotonicity assumption. Tie behavior and budget-dependent selection require validation, not an unqualified guarantee. Use: **"The stopping behavior is conservative relative to the sampled agreement criterion."** Do not say it never picks alpha too small.

Concise positioning paragraph for drafting: sampled parameter tuning and adaptive ANN search control already exist, including FAISS ParameterSpace and adaptive early-termination approaches. Our distinction is label-free, predictor-free top-k output stability applied directly to JHQ's externally specified alpha, with validation against the exhaustive sweep it is intended to replace. Do not claim sampling itself is novel or that full-grid validation is already complete. Primary literature references and precise comparisons: `[TBD]`, to be finalized in Section 5.

4.4 Calibration Cost and Reuse

Define T_cal as calibration time, T_fixed and T_rule as time per equally sized subsequent batch with fixed and selected alpha, and B as the number of subsequent batches:

T_adaptive(B) = T_cal + B*T_rule T_fixed_total(B) = B*T_fixed B* = T_cal / (T_fixed - T_rule), when T_rule < T_fixed.

For integer batches, use the smallest integer meeting the desired nonnegative/strict savings condition. If T_rule >= T_fixed, there is no finite payback. A fractional B* describes amortization, not a fractional measured workload. The comparison is conditional on the observed quality difference being acceptable; a raw gain is not automatically matched-recall gain.

Reuse assumes a stable query distribution, routing configuration, k, and index. Drift and recalibration frequency are unmeasured limitations. Recalibrating every H batches would add T_cal/H per batch under this model; no fixed percentage overhead follows merely from sampling a small fraction of queries. Current calibration timing starts after sample copying and excludes setup outside that timer; distinguish logged calibration time from a future all-inclusive deployment overhead measurement.

6 Experiments

6.1 Experimental Setup and Protocol

Keep the main text concise but include each protocol principle below, with full grids and provenance in an artifact table/appendix. Missing entries stay `[TBD]` rather than borrowing assumptions from another cohort.

- Hardware and data: the frozen project uses one RTX 5090, nominally 32 GB. Report device-available memory readings as measurements, not specifications. Cover vogue-768, arxiv-768, bge-m3, stella-trec24, openai3-1536, and openai3-3072. Exact N, preprocessing, query counts, metric and GT provenance must be reconciled with benchmark inputs `[TBD]`; dataset display names may round N.
- Recall@10, k=10 throughout the current evaluation. Alpha and k are coupled; generalization across k is unmeasured. Share query IDs, GT, distance convention and timing boundary across systems. Host queries in to host results out is the stated common boundary; record synchronization, warmups and timed repetitions per executable. The rule benchmark uses a warmup and three timed full searches (`examples/demo_jhq_alpha_fast.cu`); this does not establish every baseline's protocol.
- **Recall-QPS curves report steady-state query throughput; calibration cost is reported separately and incorporated through break-even analysis.** `AF_RESULT` times calibration and full-query throughput separately. Do not include diagnostic alpha-sweep QPS in frontiers; its diagnostic readbacks change timing.
- Mandatory CPU JHQ: strongest defensible reproducible setup, source revision, thread count, affinity/pinning, training budget, matched recall and timing. Attempt the original authors' artifact; if the available baseline is `JHQ_repro`, identify it as a reimplementation rather than the authors' executable. Frozen artifact notes document build incompatibilities and unmatched residual training. The historical 32-thread versus 208-thread advantage is a protocol warning, not a finalized GPU speedup. CPU speedup: `[TBD]` until rerun with matched training and a defensible pinned configuration. Explicitly disclose absence if unresolved.
- Main GPU baselines: IVF-RaBitQ, CAGRA and IVF-PQ. Optional references: IVF-Flat, brute force, primary-only/JQ-like hierarchy ablation. Record source/library revisions, build and search grids, nlist, nprobe, PQ code size, RaBitQ mode, CAGRA graph/build/search parameters and precision. State memory/code-budget matching or explicitly acknowledge differing budgets. Final baseline grid/provenance table: `[TBD]`.
- Current JHQ FIX/RULE grid in `data/paper_fronts.log`: nprobe={8,32,128,256,512,1024}; (M,nlist) are vogue (96,4096), arxiv (96,8192), BGE (128,32768), stella (128,32768), OpenAI-1536 (192,8192), OpenAI-3072 (384,4096). These are observed settings, not proof of exhaustive tuning. Retain QUANT4 evidence and its frozen source rather than silently choosing a slower RaBitQ mode.
- Frontier selection follows the nondominated envelope in `figures/style.py`. QPS at a stated recall is log-linearly interpolated between adjacent retained measured points; label resulting ratios interpolated, and never extrapolate beyond either method's measured range. Publish exact source points and parameter configurations. A sweep endpoint is not an architectural recall ceiling.
- Disclose coarse-training history: earlier configurations included roughly six training points per centroid; later work used much denser training, around 39. `results/front6/README.md` and `NTRAIN.md` distinguish training regimes. Freeze

the exact training recipe/cache provenance for every plotted cohort `[TBD]`; never combine pre-fix and post-fix JHQ points as one final frontier.

- Disclose Vogue's imbalanced coarse list as a characteristic of the observed trained index, without assigning an unproven cause or dismissing poor performance. README reports 135,489/932,328 vectors, about 14.5%, but its cited `data/export_ivf.log` is absent from the current frozen directory. Freeze raw histogram evidence `[TBD]` before publishing that measured percentage.
- Repeatability: `results/front6/README.md` reports cold-cache Vogue recalls 0.9852/0.9842/0.9849 with the same candidate set. Treat changes below roughly 1e-3 as within observed build-to-build variation unless repeated evidence resolves them. This is an observed range, not a confidence interval or a universal noise threshold. Raw repeated-run evidence and systematic timing uncertainty remain `[TBD]` to freeze.

6.2 End-to-End Recall-QPS — RQ1

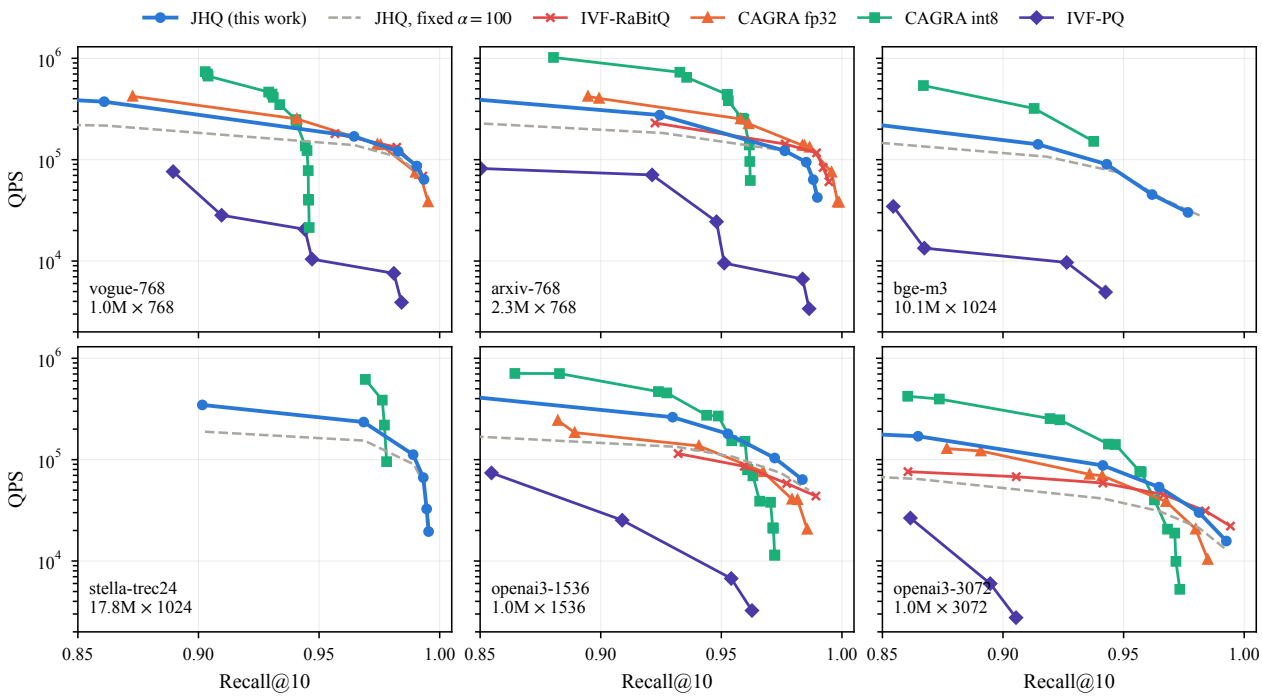


Figure 4: **fig_frontier.** six-panel recall-QPS frontier *Full caption material and every caveat: Part II,* `fig_frontier.py`.

Use `fig_frontier` to compare operating regions on the common recall interval. Evidence sources are `data/paper_fronts.log`, `data/paper_rabitq.log`, `data/bench_quant.log`, and **`report_adc2026/v47/fronts.json`**. The last source supplies older CAGRA/IVF-PQ sweeps, not current JHQ points. Verify protocol compatibility before treating the combined figure as a finalized comparison.

Draft around dataset-specific regions: JHQ can be strong at moderate/high recall, particularly on the higher-dimensional OpenAI workloads; RaBitQ can catch or pass it in high-recall tails. Quantify only interpolated matched-recall comparisons supported by overlapping curves. No universal winner or causation from dimension alone.

CPU JHQ belongs in a compact matched-recall table with source and protocol, not an optional footnote; current entries are `[TBD]`. Keep CPU throughput separate from the GPU frontier axes. Allocation failures must name the measured configuration and library path; `data/rabitq_bigsets.log` and `data/paper_rabitq.log` support limited failure disclosures, not a claim that the algorithm can never index those datasets. CAGRA precision variants must be identified individually.

6.3 Adaptive Refinement Evaluation — RQ2

6.3.1 Variation in Sufficient Alpha Use `fig_alpha` and the Br=8 `alpha6.log` sweep at nprobe=128. Show the grid and right-censored arxiv endpoint; use "4 to at least 200." The current plotted quantity is **ivf_recall - recall**, so "ranking loss" is appropriate with that explicit definition: routing loss has been removed. If changing to 1-Recall@10, label it recall loss/error instead. Do not use diagnostic QPS as performance evidence.

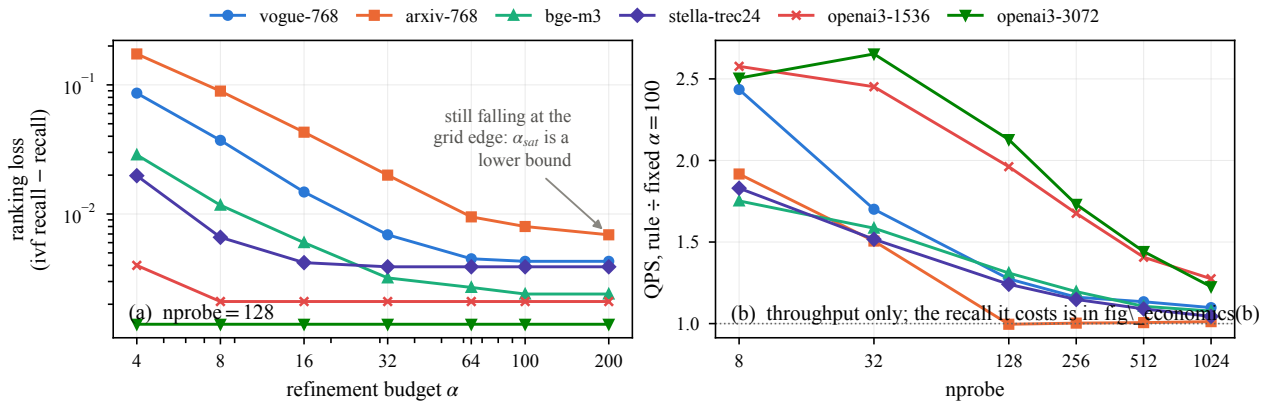


Figure 5: **fig_alpha.** ranking loss vs alpha; what the rule is worth *Full caption material and every caveat: Part II,* `fig_alpha.py`.

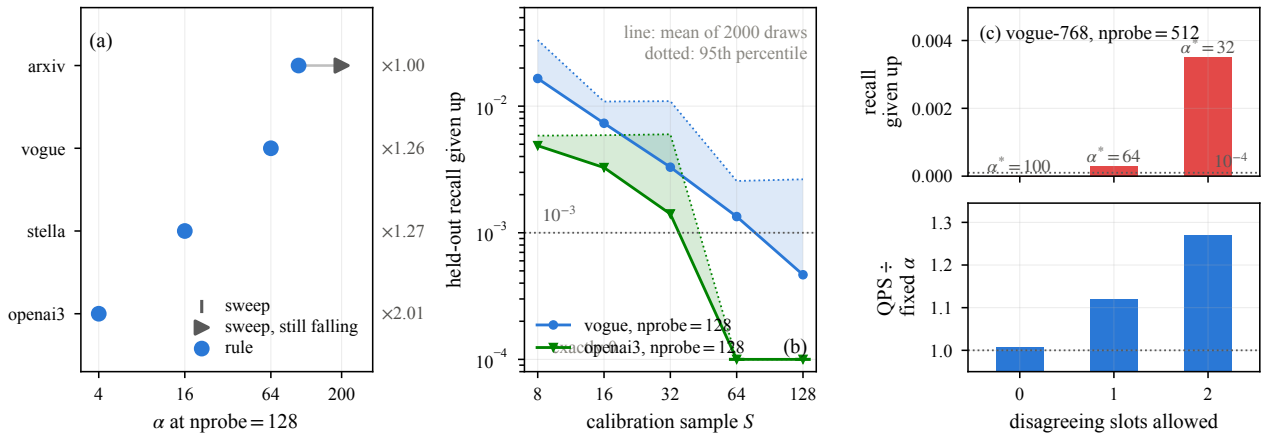


Figure 6: **fig_calibration.** the rule against the sweep; sample size; tolerance *Full caption material and every caveat: Part II,* `fig_calibration.py`.

6.3.2 Calibration versus Exhaustive Sweep Use `fig_calibration` with references keyed by **(dataset,nprobe)**. Current panel (a) compares four sample-rule configurations at nprobe=128; nprobe=512 has no corresponding exhaustive sweep and must not reuse another nprobe's answer. The script derives swept alpha using a 3e-4 tolerance to the largest-grid ranking loss. That operational threshold is smaller than the observed inter-build range; do not describe it as an established noise floor. Re-evaluate sensitivity to the plateau definition `[TBD]`.

`data/alpha_sample.log` supports sample-size sensitivity, including OpenAI-3072 at S=8 selecting alpha=2 and losing 0.0058 recall. However, this older implementation uses a fractional tolerance; it is not a complete sensitivity study of the final integer-slot rule. `data/alpha_fast.log` supports epsilon={0,1,2} slot experiments; distinguish linear and bisection rows rather than letting a parser overwrite repeated configurations. For Vogue at nprobe=512, the two-slot bisection row selects 32 and loses 0.0035 recall. Do not generalize one dataset's tolerance choice to all workloads.

Required final validation `[TBD]`: rerun final-policy sample-size and slot sensitivity, compare selected alpha and full-batch recall against the exhaustive grid at each tested (dataset,nprobe), document alpha_max censoring, repeat samples/builds, and check agreement monotonicity and tie behavior. Separate selection accuracy from speed and from reference quality.

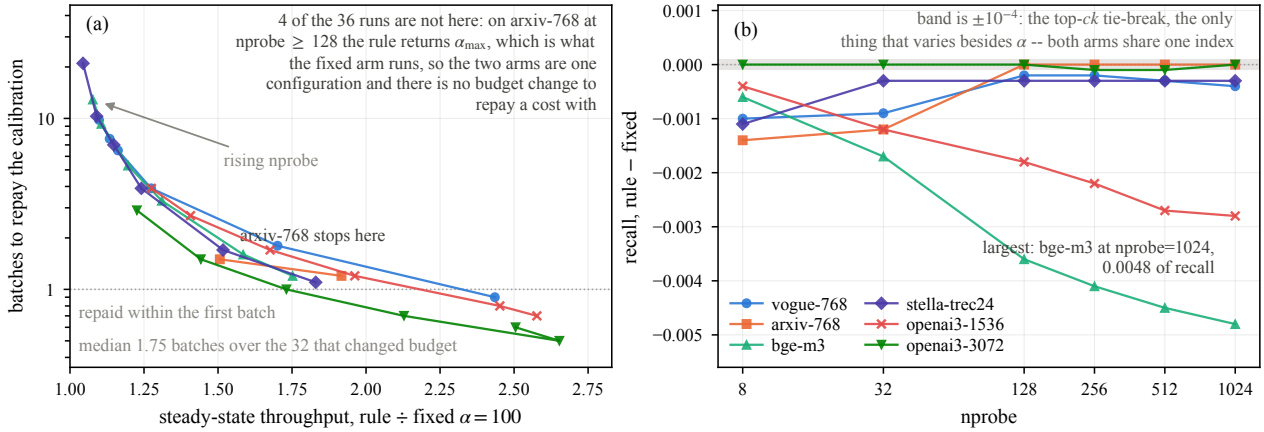


Figure 7: **fig_economics.** what the calibration costs, and when it is repaid *Full caption material and every caveat: Part II,* `fig_economics.py`.

6.3.3 Steady-State Gain and Break-Even Make economics a headline result, using all RULE rows and their paired within-row `qps_max`, not unrelated FIX timings. Include selected alpha, steady-state gain, recall delta, calibration ms, and batches_to_repay. The table below was recomputed directly from frozen `data/paper_fronts.log`:

Quantity	Verified value and interpretation
RULE configurations	36
Logged calibration time	2.0–47.5 ms
Finite logged payback range	**0.5–178.6 batches**, correcting the review's 0.6 minimum
Median finite payback	2.7 batches over 35 finite rows; exclude the -1 sentinel
Logged finite rows requiring more than two batches	18 of 36 total configurations; the additional non-repaying row never amortizes
No finite payback	arxiv, nprobe=128: gain=0.996, sentinel=-1.0
Largest raw gain	2.653 at OpenAI-3072, nprobe=32; logged payback=0.5
Longest finite payback	arxiv, nprobe=256: gain=1.003, payback=178.6
Largest logged recall loss	BGE, nprobe=1024: 0.0048

BGE losses also exceed 0.003 at nprobe=128 (0.0036), 256 (0.0041), and 512 (0.0045). Therefore do not repeat the README's "all except one <=0.003" or label all raw gains equal-recall. Sub-1e-3 deltas such as -0.0003 are within observed build-to-build variation; larger losses remain visible.

Finalize **`fig_economics` [TBD: review and integration]**: x=steady-state QPS gain, y=logged finite batches_to_repay on a log scale, color/marker identifying dataset and optionally nprobe. Add gain=1 and payback=1 reference lines. Represent the non-repaying configuration separately with an explicit annotation; never plot -1 on the log axis or silently drop it. All 36 configurations must be accounted for. An untracked economics script and rendered outputs appeared during this audit through concurrent workspace work; they are not part of this skeleton commit. Its current "one real cost" annotation and claim that only one recall delta lies outside the observed variation band conflict with the log and require correction. Caption specifies the 35-row finite median, rounding, quality deltas and stable-workload assumption.

Interpret jointly: substantial gains tend to repay quickly; negligible gains produce long or no repayment. A finite 178.6

is long, not mathematically "never," and tiny timing differences require repetition. The stale "eight batches / 8000 queries" note is not the current result. Economics measures conditional workload reuse, not observed generalization across future drifting batches.

6.4 Representation and Execution Ablation — RQ3

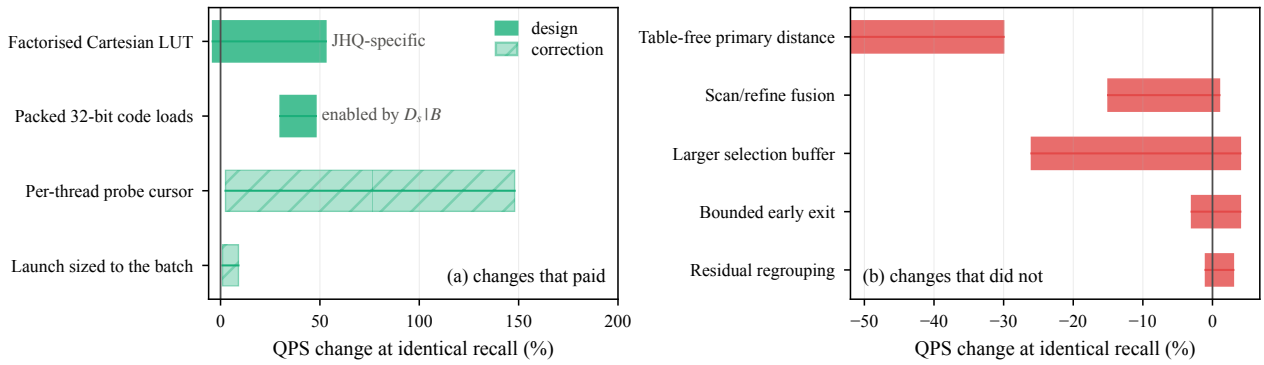


Figure 8: **fig_ablation.** what paid, and what did not *Full caption material and every caveat: Part II,* `fig_ablation.py`.

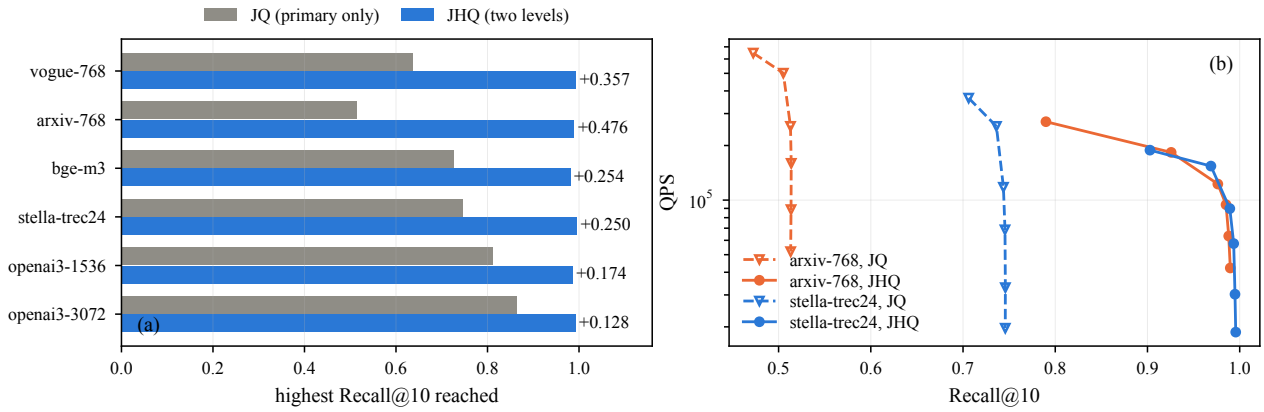


Figure 9: **fig_hierarchy.** what the second level buys *Full caption material and every caveat: Part II,* `fig_hierarchy.py`.

6.4.1 Value of the Residual Hierarchy Use `fig_hierarchy`, `data/hierarchy_ablation.log` and the full JHQ frozen frontier. Compare primary-only/JQ-like versus full JHQ under matched routing/training. Report attained recall across the sweep and work added by refinement. Call maxima "highest measured recall," not architectural ceilings. This ablation tests the hierarchy, not a fully tuned independent JQ system.

6.4.2 Cartesian-Factorized LUT Isolate full versus factorized primary tables with identical codes, candidates and policy. State exact representation equivalence, then measured construction/search effects. Historical evidence is under `results/v47_split_lut/`; freezing a matched final-implementation ablation and uncertainty is `[TBD]`. Do not substitute the algebraic entry ratio for a speedup.

6.4.3 Representation-Enabled Optimizations and Engineering Separate packed access, physical layout, and cursor/launch corrections by status. `fig_ablation` should not visually present them as equally novel. Historical packing and correction evidence is in `results/front6/NEGATIVES.md`, `results/front6/v52.log`, `results/front6/v51.log`, and `data/v57_launch.log`; final matched ablation matrix is `[TBD]`. Do not add percentage gains from different cohorts to infer a combined speedup. Report actual effect sizes side by side even if a simpler optimization has the larger effect.

6.5 Mechanistic Analysis and Negative Results — RQ4

Use `fig_negatives`; discuss larger selection buffers, regrouping, scan/refinement fusion, early exit, and table-free distance. Evidence: `data/v54.log`, `data/qdup.log`, `data/qdup_stella.log`, with internal experiment descriptions in `results/front6/NEGATIVE`

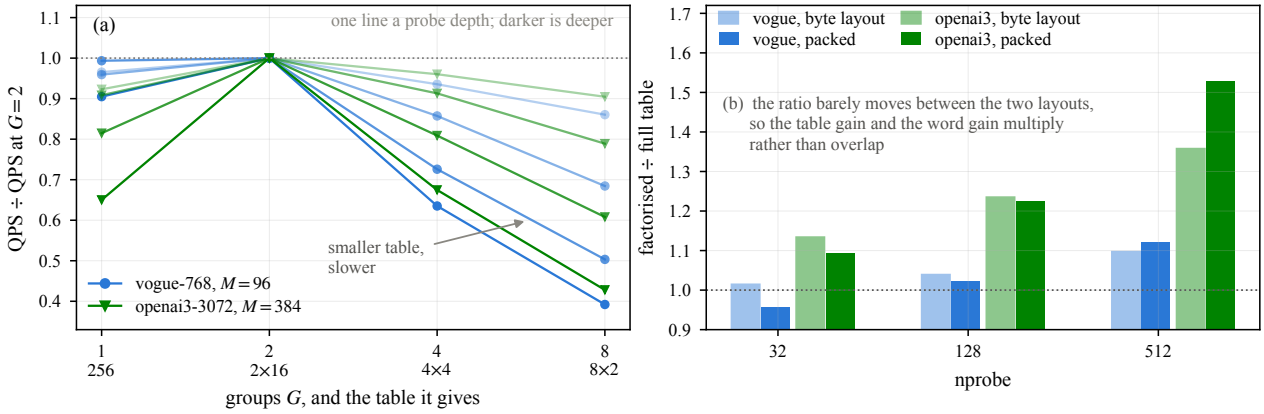


Figure 10: **fig_lutgroups. why halves; and the table x layout 2x2** *Full caption material and every caveat: Part II,* `fig_lutgroups.py`.

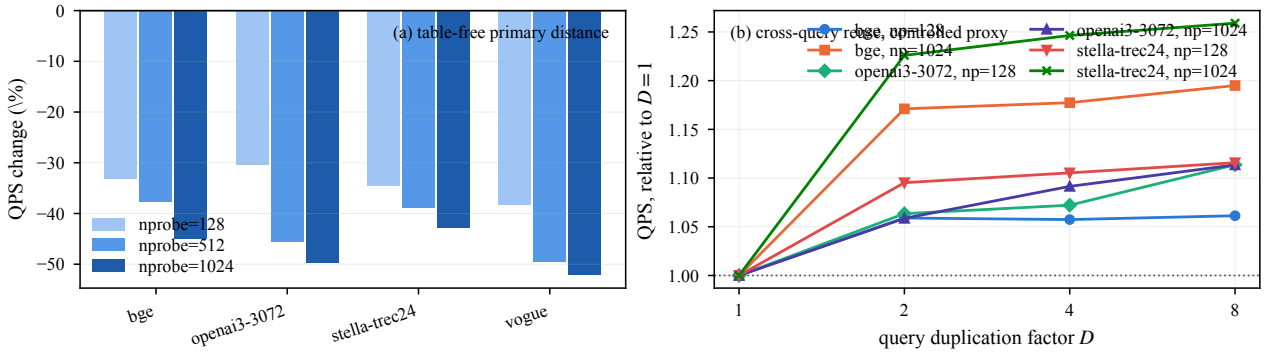


Figure 11: **fig_negatives.** the table-free distance, and reuse as a controlled proxy *Full caption material and every caveat: Part II,* `fig_negatives.py`.

`V54_SIGN_IP.md`, and `QDUP.md`. Freeze other raw negative-result cohorts before quoting numerical ranges `[TBD]`.

The table-free experiment shows that fewer table bytes need not mean faster execution. Instruction count, occupancy, cache, shared-memory residency and memory traffic all matter. Separate measured runtime/resource counts from causal explanations requiring profiling. One-card observations do not establish a cross-architecture law.

Duplicate-query experiments are a **controlled reuse proxy**, not an upper bound on another scheduler. If L2 reuse is inferred, say the pattern is "consistent with L2 already capturing much of the reuse." Direct attribution and selection-stage profiling remain `[TBD]`. Prefer shortening setup before removing this evidence; if necessary fold negative results into the ablation discussion without converting the experiment organization to version history.

6.6 Batch Sensitivity — RQ5

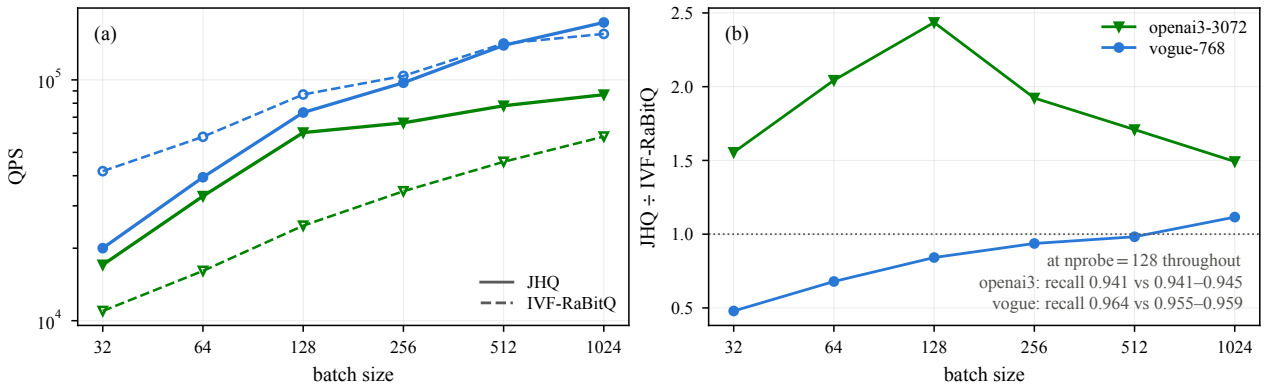


Figure 12: **fig_batch.** throughput and the JHQ/RaBitQ ratio against batch *Full caption material and every caveat: Part II, `fig_batch.py`.*

Use `fig_batch` and `data/batch_sweep.log`, explicitly labeled **batch sensitivity at fixed nprobe=128**. The logged sweep covers OpenAI-3072 and Vogue with batch labels from 32 to 1024. Show both recalls alongside throughput ratios: at the largest logged batch, OpenAI-3072 recalls are 0.9414/0.9436 and Vogue recalls are 0.9645/0.9586 for JHQ/RaBitQ. These are not matched-recall winner comparisons.

Reconcile logged batch=1024 with actual query-file row counts and benchmark batching before describing the workload as 1024 independent queries `[TBD]`. Do not manufacture an independent 10k workload by duplicating approximately 1000 real queries. A matched-recall batch sweep, independent larger query set and causal attribution of batch effects remain `[TBD]`. Do not extrapolate curves to another GPU or batch size.

6.7 Build Time and Memory — RQ6

Use `fig_cost` and `fig_memory`. `data/vram.log` measures RaBitQ resident GPU memory on four datasets; `data/paper_fronts.log` measures JHQ. State measurement point (index plus allocated search workspace), units and nprobe/workspace configuration. For example, the nprobe=8 JHQ / measured RaBitQ values are Vogue 1405.2/1012.0 MiB and OpenAI-3072 4047.2/3324.0 MiB. Measured JHQ residency varies with nprobe; the current figure loader retains the last FIX row, so do not mix its bars with first-row prose values.

Model primary/residual codes, IDs/corrections, centroids and query/candidate workspace separately from measured totals. Add **other / allocator / runtime workspace = measured minus modeled**. This residual segment reconciles the accounting but does not independently measure its causes; a negative gap signals a model error. Report competitor components only where measured. RaBitQ has the smaller observed resident footprint on the four measured datasets. Say "JHQ occupies a different memory-accuracy-throughput regime," not universal memory superiority.

For build time, separate training from encoding, cold training from cached loads, and input loading from index construction. `fig_cost` selects the first FIX row for build cost and converts logged milliseconds to seconds. Cold-cache provenance, units and equivalent competitor timing boundaries need final verification `[TBD]`. Do not treat later cached training rows or unmatched CPU training as full build comparisons.

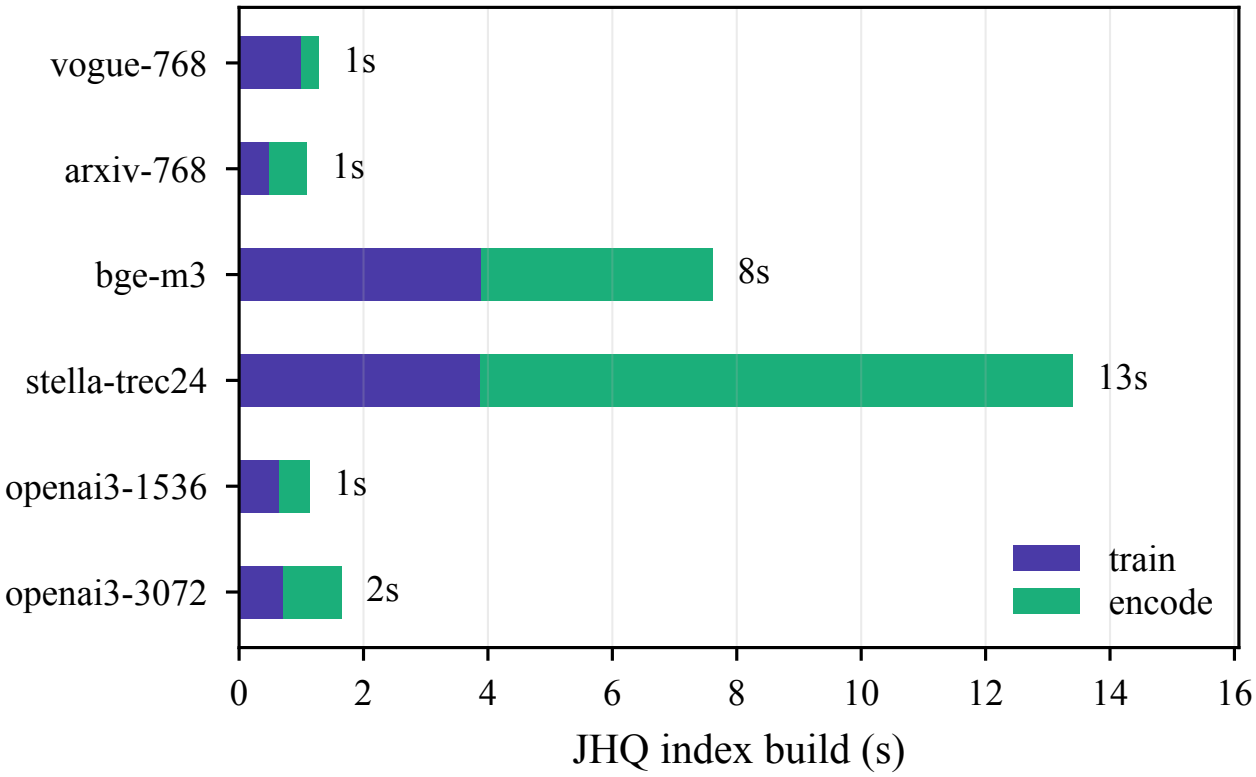
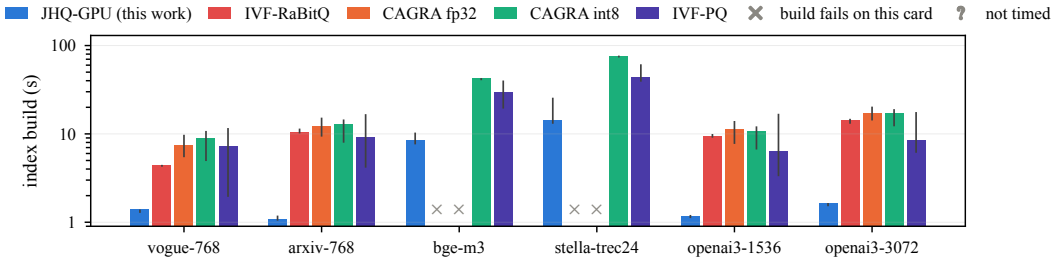


Figure 13: **fig_cost.** JHQ's build, split into training and encoding *Full caption material and every caveat: Part II,* `fig_cost.py`.



whisker: the range across swept configurations and repeat builds, not a confidence interval. JHQ's bar is its one cold training plus its median encode: training is cached after a dataset's first row, encoding is not and re-runs every row. All indexes train the coarse quantiser at 39 points a centroid.

Figure 14: **fig_build. index build time, all five methods** *Full caption material and every caveat: Part II,* `fig_build.py`.

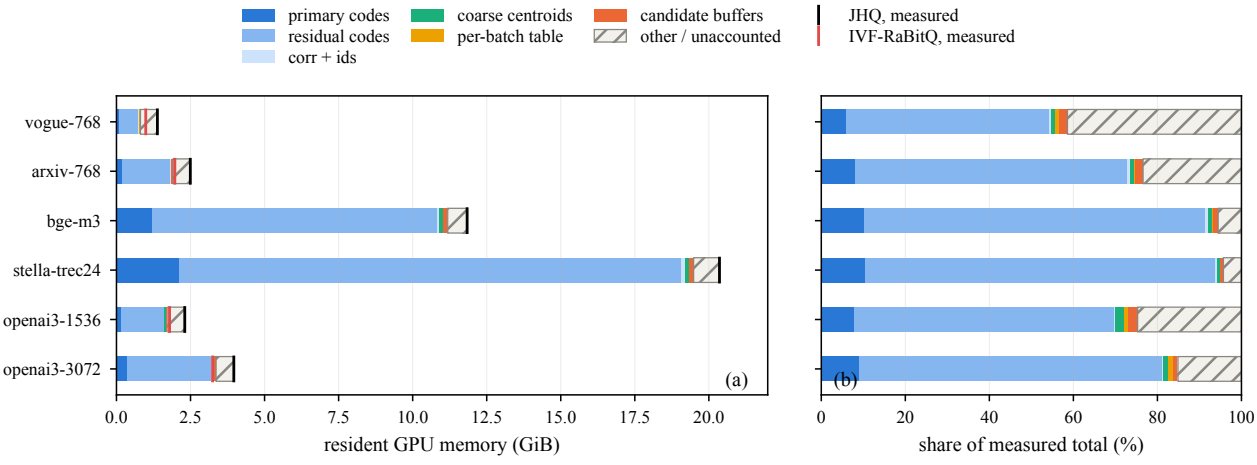


Figure 15: **fig_memory.** where JHQ\'s memory goes, and why it is above RaBitQ\'s *Full caption material and every caveat: Part II,* `fig_memory.py`.

6.8 Experimental Takeaways

Conclude only what the finalized evidence supports: Cartesian structure enables exact reorganization of primary-distance evaluation; refinement is valuable but its useful budget depends on workload; output-stability selection must be judged by both quality and payback; competitive throughput occupies dataset/recall/batch regions with explicit memory and build tradeoffs. CPU acceleration claims remain `[TBD]` until the mandatory baseline lands.

Figure and artifact completion map

This task creates only the mother framework. Existing scripts and data are preserved; the following are drafting/experiment deliverables, not claims that they have been completed.

Figure	Placement	Remaining check or action
fig_pipeline	3.1	Consistent Q, actual training dependencies, no equation numbers.
fig_layout	3.2 / 3.3.2	Separate coalescing from packed instruction reduction; no cache-line waste myth.
fig_lut	3.3.1	Exact current representation; replace hard-coded equation references.
fig_rule	4.3	Align set-slot criterion, actual sample selection and bisection/linear mode.
fig_frontier	6.2	Audit cohort provenance, tuning and hard-coded RaBitQ points in style.py against bench_quant.log. No ceiling shading.
fig_alpha	6.3.1	Current ranking-loss axis is valid; stale “25x” and equal-recall/gain captions require correction.
fig_calibration	6.3.2	Preserve dataset+nprobe keys; distinguish old fractional-tolerance evidence, repeated modes and plateau threshold sensitivity.
fig_economics (new working-tree draft)	6.3.3	Review/integrate scatter, correct recall-loss overclaim, and distinguish non-repayment from finite y values.
fig_hierarchy	6.4.1	Highest observed recall, matched setup, no universal ceiling.
fig_ablation	6.4	Separate novelty tiers and freeze final matched ablations.
fig_negatives	6.5	Controlled reuse proxy; measured facts versus inferred mechanism.
fig_batch	6.6	Fixed nprobe, both recalls; reconcile actual query counts.
fig_cost / fig_memory	6.7	Cold-build provenance, measurement configuration, model reconciliation and measured competitor values.

Use text at least approximately 7 pt at final print size, embedded readable fonts and distinguishable markers. Caption and script comments must agree with the plotted quantity. Earlier reviews saying all figure fixes are complete do not supersede current script inspection.

Evidence readiness and remaining experimental TODOs

Claim	Current readiness	Required before final prose
Exact Cartesian table factorization	Semantics/algebra complete; implementation precondition checked in `jhq_v57_launch_nq/jhq_gpu_index.cu`	Final isolated timing ablation is separate.
Alpha variation and censored endpoint	Frozen diagnostic recall evidence complete within tested grid (`alpha6.log`)	State nprobe and plateau convention; no diagnostic QPS.
Calibration economics summary	Frozen row-level arithmetic complete (`paper_fronts.log`)	Finalize economics figure; qualify timing scope, quality and stationarity.
Final policy universally recovers sufficient alpha	**Blocked [TBD]**	Full matched sweep, final slot-rule sensitivity, sample/build repetition and censoring checks.
GPU operating-region comparisons	Frozen curves exist; protocol completion pending	Full baseline grids, training/cache lineage, interpolation provenance and matched-recall tables.
Faster than original CPU JHQ	**Blocked [TBD]**	Reproducible strongest CPU setup, explicit source, matched training/recall and pinned timings.
Residual hierarchy value	Frozen ablation exists	Audit matching and avoid calling sweep maxima ceilings.
Memory tradeoff	Measured totals available for JHQ and four RaBitQ datasets	Align workspace configuration and modeled/observed accounting.
Fixed-nprobe batch sensitivity	Frozen measurements available	Verify batch construction; matched-recall batch conclusions remain blocked.
Generalization across k, drifting workloads, GPUs	**Unmeasured [TBD]**	Run relevant studies or explicitly retain limitations.
Vogue imbalance and repeatability numbers	Documented in notes, frozen raw provenance incomplete	Freeze histogram and repeated-run evidence before numerical publication.

Priority order: finalize mandatory CPU and baseline protocol; validate the final alpha policy and quality exceptions; finalize economics figure; freeze matched representation/execution ablations and build/measurement provenance; resolve batch counts, repeatability and diagnostic evidence. Larger-k, larger independent batches, drift and a second GPU may remain explicit scope limitations if not measured. No experiment data or implementation should be changed to make a narrative claim true.

Part II — figure reference

Each figure at full width, then its script's docstring verbatim: what it measures, which runs are in it, which are excluded and why, and which noise floor applies. This is caption material.

`fig_pipeline` the JHQ-GPU pipeline, shaded by what changed

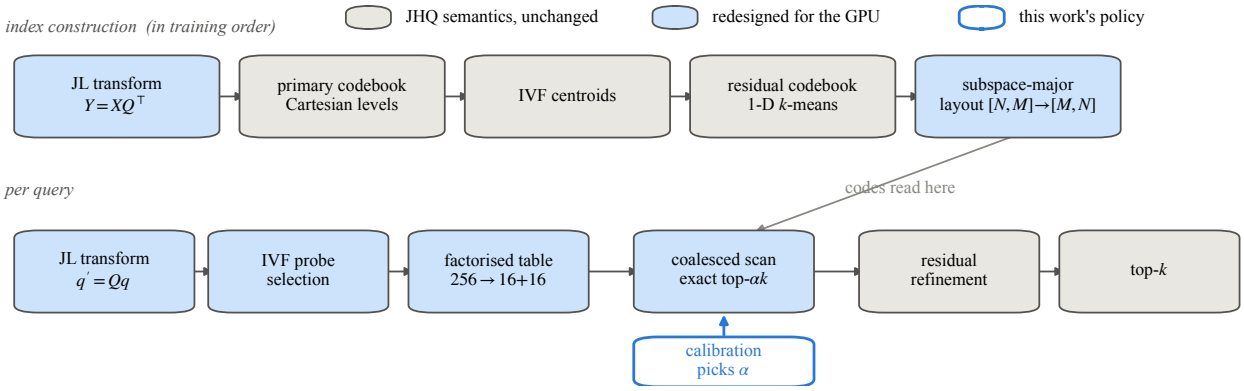


Figure: the JHQ-GPU pipeline.

Section 3.1. Two rows: what happens once at build time, and what happens per query. The point of the drawing is which stages are GPU-native redesigns and which are unchanged JHQ semantics, so the two are shaded differently rather than described in a caption.

The build row is in the order the code actually trains in (the JHQ_TRAIN_PHASE sequence in jhq_gpu_index.cu): the IVF centroids are fitted before the residual codebook, not after it. The residual codebook is trained on residuals drawn from a sample, so it does not wait on every vector being assigned -- drawing it the other way round would suggest a dependency that is not there.

Q is the JL matrix. The paper writes the primary levels as a Cartesian product of one-dimensional Lloyd-Max levels; the equation number is left to the text rather than baked into the drawing.

`fig_lut` the Cartesian factorisation, and what it saves per query

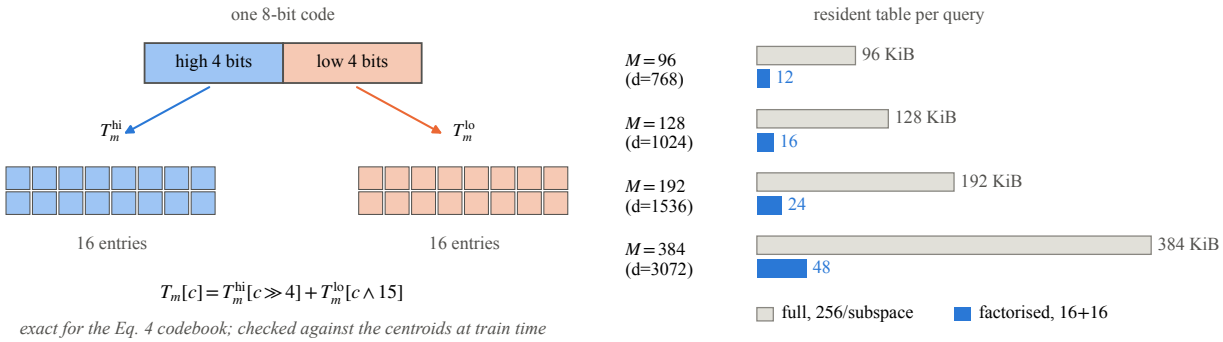


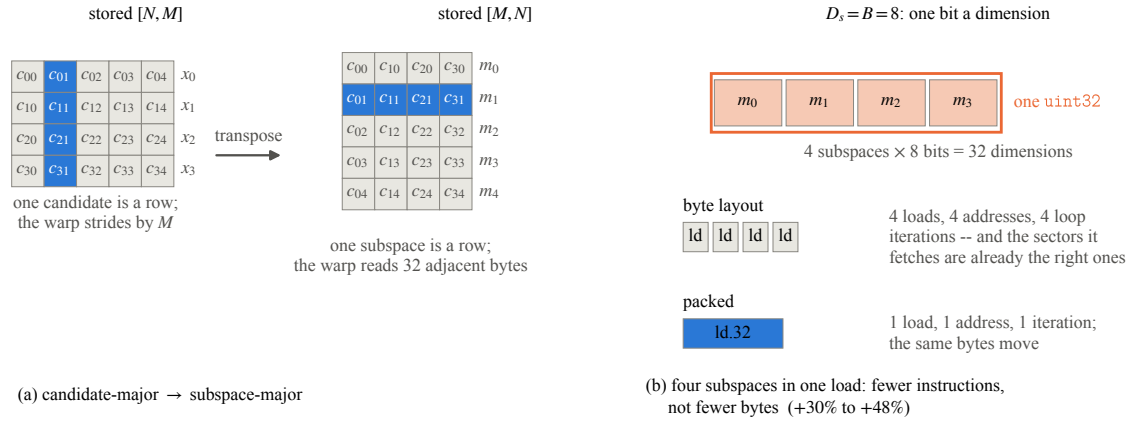
Figure: the Cartesian factorisation of the primary distance table.

Section 3.3.2. Equation 4 builds the K codewords as a Cartesian product of the per-dimension levels of equation 3, so the high nibble of a code fixes the first Ds/2 coordinates of its centroid and the low nibble the rest. The table splits exactly:

```
T_m[c] = T_m^hi[c >> 4] + T_m^lo[c & 15]
```

256 entries a subspace become 16 + 16. The identity is exact for this codebook and is verified against the centroids at train time, not assumed -- a Lloyd-refined product quantiser does not have the property and the check throws rather than returning wrong distances.

`fig_layout` the transpose, and four subspaces in one load




```
Figure: the physical layout of the primary codes, and what a warp reads.

Section 3.2.3 and 3.3.3. Two changes to the same bytes, neither of which
touches a distance.

(a) Candidate-major to subspace-major. Stored [N, M], the 32 threads of a warp
    scanning subspace m read one byte each, M bytes apart -- 32 separate
    transactions. Stored [M, N] the same 32 bytes are adjacent.

(b) Four subspaces in one 32-bit word. Ds = B = 8 makes the primary code one
    bit a dimension, so four subspaces are 32 dimensions and fit a uint32.

    This is not a coalescing fix and does not move fewer bytes. Since Pascal a
    global access is served in 32-byte sectors, so the byte layout in (a) was
    already reading exactly the sectors it needed. What the packed load
    removes is instructions: one load, one address computation and a quarter
    of the loop iterations where there were four. The scan issues far more
    instructions than its arithmetic needs -- which is also why the table-free
    distance in Section 6.5 loses despite moving less memory -- so removing
    them is where the +30% to +48% comes from.

No candidate's distance changes; the same codes are read in a different order.
```

`fig_rule` how the budget is chosen, and one real calibration

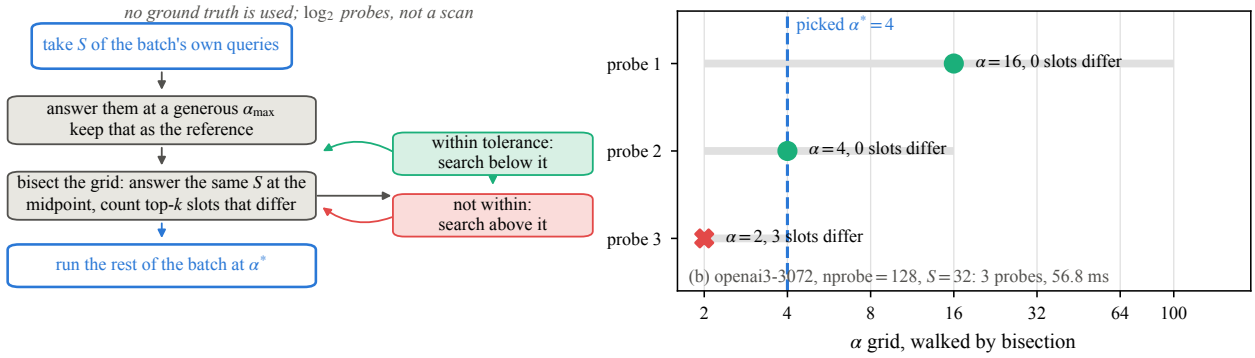


Figure: how the refinement budget is chosen, and one real calibration.

Section 4.3. The rule asks the only question the system can answer without labels -- would a smaller budget change what I return? -- on S of the batch's own queries, and it walks the alpha grid by **bisection**, not by stepping down one value at a time.

That distinction was wrong in an earlier version of this figure, and it matters twice.

It matters for cost: three probes settle a seven-point grid instead of up to seven, which is where the calibration time in Section 6.3.3 comes from.

And it matters for what can be claimed. "Stop at the first rejection, so the choice is one-sided" describes a linear scan and is not what happens: on vogue-768 the first probe is *rejected* and the search continues upward (16 reject, 64 accept, 32 reject, picked 64). What bisection returns is the smallest grid alpha whose sampled top-k agrees within the slot tolerance, **assuming the accept predicate is monotone in alpha**.

That assumption is not free, but it is arguable rather than hoped for: exact top-ck selection gives nested candidate sets, so ck1 < ck2 implies the top-ck1 set is contained in the top-ck2 set, the refined top-k therefore moves monotonically toward the alpha_max answer, and the count of differing slots is non-increasing in alpha. Ties aside, the predicate is monotone and the bisection is sound. The paper should say this rather than assert one-sidedness.

The right panel is one real calibration read from data/alpha_fast.log -- no value in it is written into this script.

`fig_frontier` six-panel recall-QPS frontier

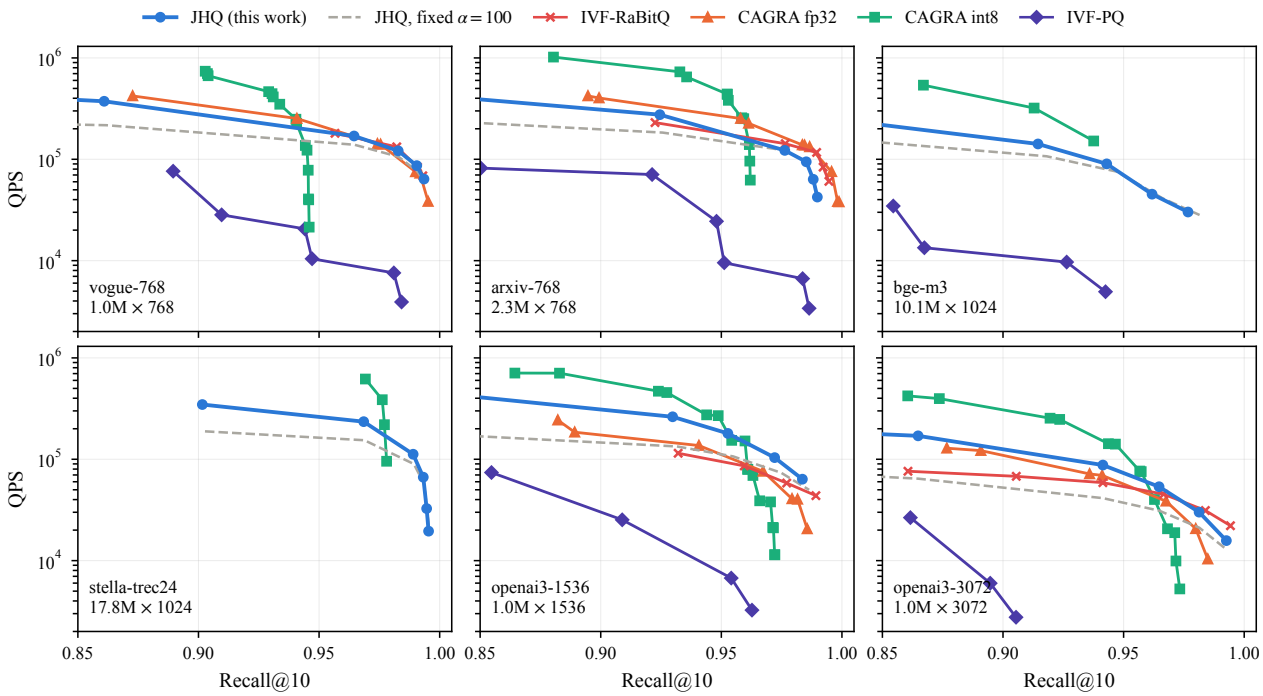


Figure: recall–QPS frontier, six datasets, every system on one RTX 5090.

Section 6.2. The dashed grey line is JHQ at the fixed alpha=100 this project used from its first run to its last; the gap to the solid line is what the calibration rule is worth.

Each curve ends where its own sweep ended, and nothing here marks a region as out of reach: a baseline whose highest swept point is below JHQ's has not been shown to be unable to go higher, only not to have been asked to. Read the curves where they overlap.

CAGRA fp32 is absent on stella and bge-m3 -- 17.8 M and 10.1 M vectors at 1024 float dimensions do not fit the card -- and IVF-RaBitQ is absent there too, for the reason given in the paper's setup. Those are the only two claims of the form "cannot", and both are allocation failures, not sweep endpoints.

`fig_alpha` ranking loss vs alpha; what the rule is worth

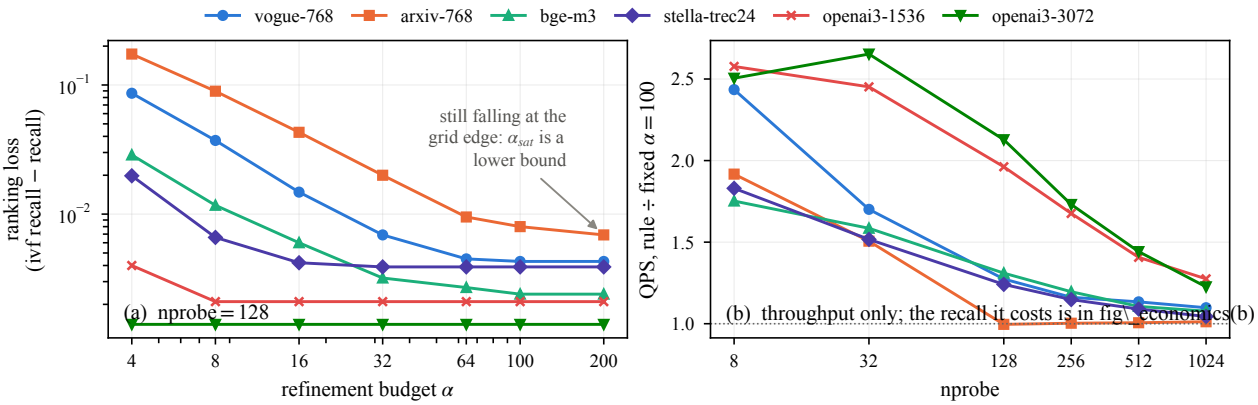


Figure: the refinement budget, and a rule that finds it.

Section 6.3, two panels.

(a) Ranking loss against alpha. The point where it stops improving spans 25x across these datasets -- flat from alpha=4 on openai3-3072, still moving at 200 on arxiv-768 -- which is why one constant cannot serve all six.
(b) What the rule is worth: QPS at the alpha it picks, over QPS at the fixed 100, at equal recall. The gain falls as nprobe rises on every dataset because alpha sizes the refinement and the scan takes over.

`fig_batch` throughput and the JHQ/RaBitQ ratio against batch

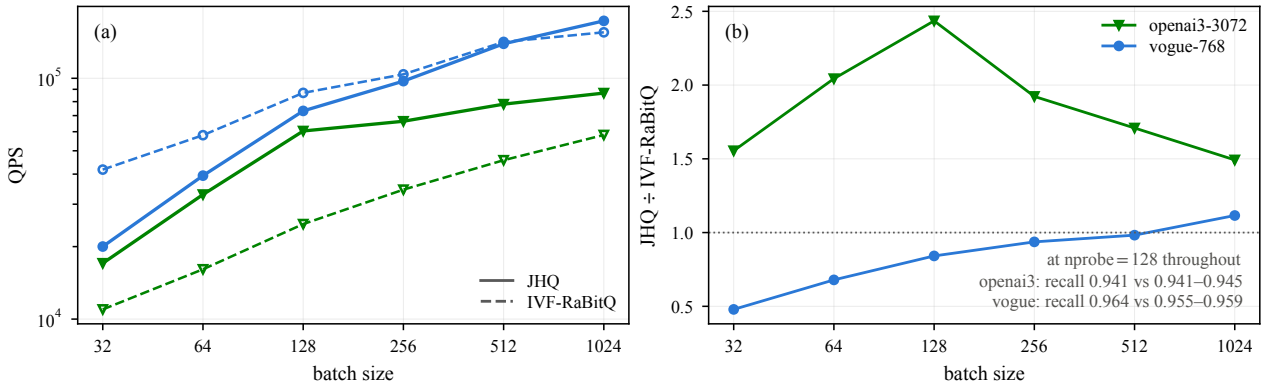


Figure: batch sensitivity at fixed nprobe.

Section 6.6. Every other number in the paper is one batch on one card, and IVF-RaBitQ's own paper reports 10^4 on a different card. This is the axis that answers the objection.

Both systems run at nprobe=128 throughout; the recall each reaches there is whatever it reaches, and is reported in panel (b). This is a sensitivity plot, not an iso-recall comparison -- the ratio is a throughput ratio at one operating point, and only panel (b)'s recall line says how comparable the two operating points are.

They are close enough to read on openai3-3072 (JHQ 0.9414, IVF-RaBitQ 0.9405 to 0.9450, so the ratio is if anything slightly generous to JHQ) and on vogue-768 they favour IVF-RaBitQ (JHQ 0.9645, IVF-RaBitQ 0.9549 to 0.9592, so JHQ is being timed at a higher recall than the system it is divided by).

The ratio is not stable in batch and moves in opposite directions on the two datasets: JHQ's lead peaks at batch 128 on openai3-3072 and is falling by 1024, while on vogue-768 IVF-RaBitQ is twice as fast at batch 32 and JHQ overtakes only at 1024, still climbing.

The sweep goes down from ~1000 rather than up to 10^4 because the query files hold about 1000 rows; duplicating them inflates throughput 4-26% through L2 reuse alone (data/qdup.log), so the comparison would be against an artefact.

`fig_ablation` what paid, and what did not

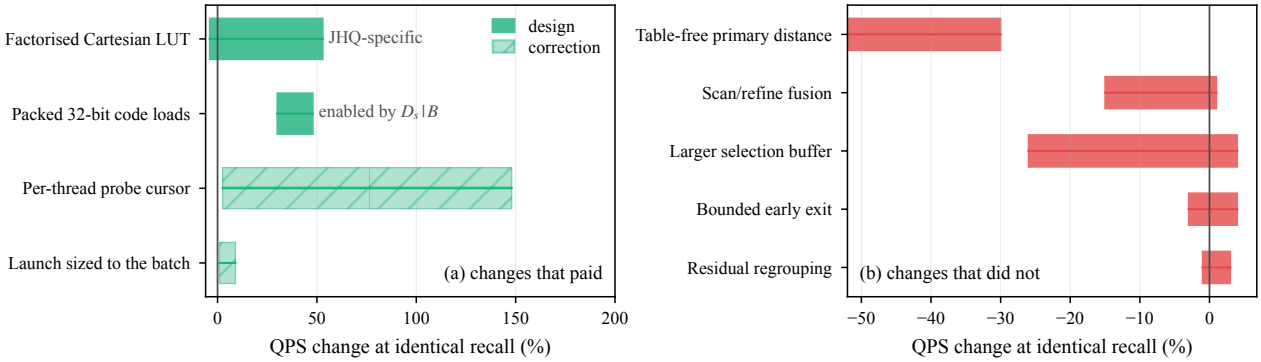


Figure: which GPU design choices pay, and which do not.

Sections 6.4 and 6.5, two panels.

(a) What each change was worth. The rows are grouped by what kind of claim they support, because an earlier version listed them flat and invited the reading that they are equally novel:

  * **JHQ-specific.** The factorised Cartesian LUT is exact only because the primary codebook is a Cartesian product of one-dimensional levels. No change to the code, the codebook or the distances. Its range is the widest here and the width is the finding, not noise: -4% at M=96, where the full 256-entry table still fits in shared memory, to +53% at M=384, where it does not. fig_lutgroups has that against M; a bar can only carry the span.
  * **Representation-enabled.** Four subspaces share a 32-bit load only because Ds | B at B=8 forces one bit a dimension. A consequence of the admissibility rule rather than an independent idea.
  * **Corrections.** These remove unintended work. They are not designs and are drawn hatched so no reader has to take the caption's word for which is which.

  Adaptive alpha is **not** on this panel. It is a policy result, its gain is measured against fixed alpha=100 rather than at matched recall, and it carries a recall cost that a bar cannot show. It belongs to Section 6.3 with fig_alpha and fig_economics, and putting it here made the largest bar in the figure the one thing in it that was not a kernel change.

(b) The five changes that did not pay. Four traded a memory saving for shared memory or registers; v54's table-free primary distance traded the other way, spending instructions to buy occupancy, and lost hardest on the one dataset whose occupancy it tripled.

Every bar is a **range across the configurations it was measured on** -- datasets and nprobe -- not a confidence interval. The runs behind different rows are not the same set, so the widths are comparable in meaning but the rows are not a single controlled experiment. The 2x2 matched ablation that would make them one (full/split LUT x byte/packed codes) is not yet run.

`fig_calibration` the rule against the sweep; sample size; tolerance

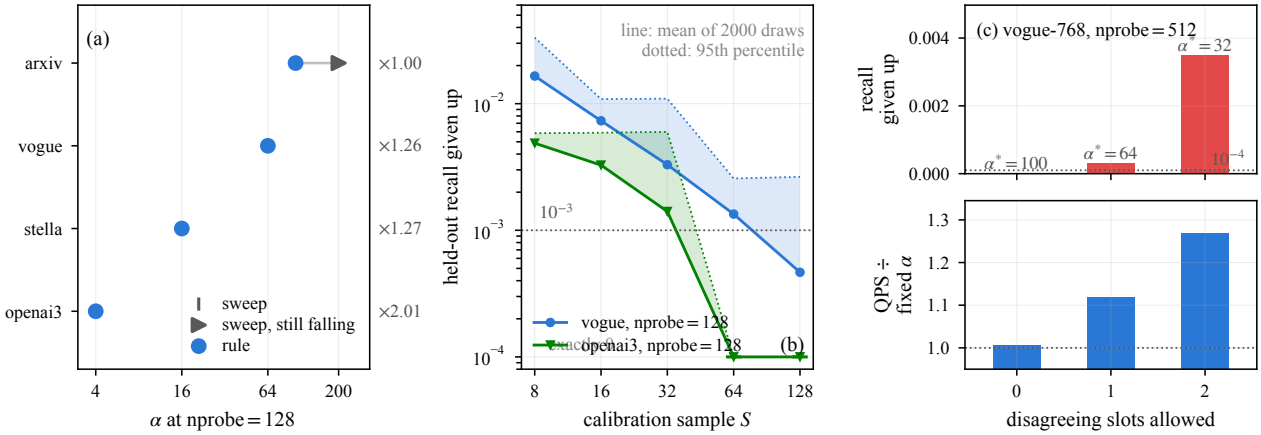


Figure: the calibration rule, validated against the sweep it replaces.

Section 6.3, the contribution's own evidence. Three panels.

(a) The alpha the rule picks against the alpha an exhaustive sweep finds, at the same (dataset, nprobe). Keyed on both: the saturation point is a property of the operating point, not of the dataset, so a sweep run at one nprobe is not a reference for a rule run at another.

That restricts the panel to nprobe=128, where alpha6.log swept the full grid {4, 8, 16, 32, 64, 100, 200} on all six datasets. The rule was also run at nprobe=512, and those four rows are not drawn, because no sweep was run at nprobe=512 on those datasets -- there is nothing to compare them against. (ALPHA_RULE.md previously reused the nprobe=128 sweep for them; that is the reuse this panel now refuses to make.)

The swept value is the smallest alpha whose ranking loss is within 3e-4 -- the build-noise floor -- of the loss at the top of the grid. Ranking loss, not recall: alpha cannot recover a neighbour whose list was never opened, so routing loss does not belong in the quantity alpha is judged on.

Three of the four are exact. arxiv-768 is the exception and is not a miss in the rule's favour or against it: its loss is still falling at alpha=200 and the rule's own alpha_max is 100, so the rule cannot reach the saturation point at all. It returns 100 and reports no gain, which is the right behaviour for a budget that is already too small.

The grid is walked by bisection, so there is no "stop at the first rejection" and no one-sidedness to claim -- see fig_rule. What the rule returns is the smallest grid alpha its sampled criterion accepts, bounded above by alpha_max. The sample estimates the true saturation point; it does not bound it, and arxiv-768 is what the difference looks like.

(b) Sample size, resampled. This panel used to plot one calibration draw per S and read a knee off it, which is how the paper came to recommend S=32. That recommendation does not survive being measured properly.

With the full alpha grid dumped a query at a time on a frozen index (data/alpha_perquery/), every draw can be replayed offline: sample S queries, run the rule on exactly what it would have seen, and score the alpha it picks **on the queries it did not see**. 2000 draws a cell, alpha_resample.py, no GPU.

The single draw was lucky. On vogue-768 at nprobe=128 it picked alpha=64 and gave up 0.0002. Across 2000 draws at the same S=32 the modal pick is 32, not 64; the mean held-out loss is 0.0033, sixteen times larger; the 95th percentile is 0.0110, a full point of recall; and only 27% of draws land inside 1e-3.

S=32 is enough on openai3-3072, where the saturation point is alpha=4 and 76% of draws are inside 1e-3, rising to 98% at S=64. It is not enough on vogue-768, where the loss curve is still falling at S=128 and only 89% of draws are inside 1e-3 there. The knee is a property of the dataset, and the paper has to say S per workload with its measured risk rather than

```
    print one constant.
(c) Tolerance, on vogue-768 at nprobe=512, all three points from the
    bisecting rule. The reference is 1e-4, not 1e-3: both arms of each
    comparison run on one index with one set of codebooks, so the only thing
    that varies besides alpha is the top-ck tie-break. The 1e-3 figure quoted
    elsewhere is a build-to-build floor and does not apply to a paired
    comparison.
```

`fig_economics` what the calibration costs, and when it is repaid

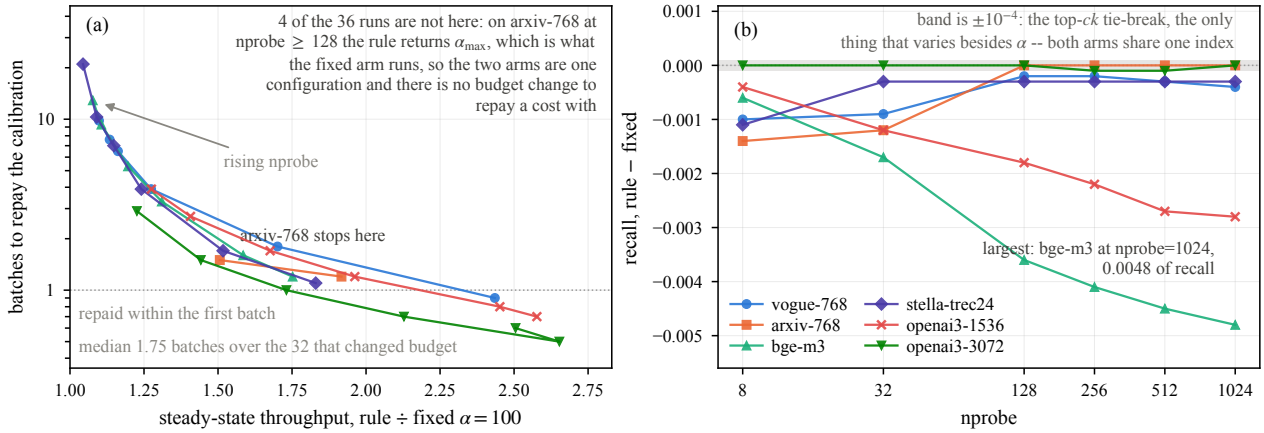


Figure: what the calibration costs, and when the workload has repaid it.

Section 6.3.3. Every other number for the rule is a steady-state throughput gain, measured after alpha has been chosen. The choosing is not free: it answers S=32 of the batch's own queries at several alpha before the batch runs, and that cost is paid once and recovered over the batches that follow. Reporting the gain without it would be reporting half a trade.

(a) The relation between the two. Each dataset is one path through its six nprobe values, from nprobe=8 at the lower right -- large gain, repaid inside the first batch -- to nprobe=1024 at the upper left. The direction is the same on all six because both ends of the trade move with nprobe: a larger probe list puts more true neighbours in the candidate pool, so the saturation budget rises and the gain over a fixed alpha=100 shrinks, while the calibration itself gets more expensive (2.0 ms to 47.5 ms) for exactly the same reason.

So the cost is largest where the gain is smallest. That is the honest shape of this contribution, and it is better said in a figure than left for a reader to derive: the rule is worth running in the regime where alpha can fall a long way, and is close to free-and-pointless where it cannot.

Four configurations are not on this panel at all, and the reason matters. On arxiv-768 at nprobe 128, 256, 512 and 1024 the rule returns alpha=100, which is alpha_max, which is also what the fixed arm runs. The two arms are then *the same configuration*: demo_jhq_alpha_fast.cu times full(picked) and full(grid[0]) separately, three repeats each, so with picked == grid[0] the reported gain is the ratio of two 3-run means of one thing. It comes out 0.996, 1.003, 1.006 and 1.012 -- repeat-timing variation of about 1.2% -- and batches_to_repay then divides the calibration cost by that noise, giving 179, 108 and 58. Those are values of 1/noise, not economics, and the recall delta on all four is exactly 0.0000, which is the same fact seen from the other side.

So they are excluded, and the correct statement about them is not "slow to repay" but **the budget did not change**: arxiv-768's saturation point is above the rule's own alpha_max (see fig_calibration), the rule returns alpha_max, and there is no steady-state saving for a calibration cost to be recovered from.

(b) The other half of the trade: what the chosen alpha costs in recall.

The band is +-1e-4, and choosing it correctly matters. The 1e-3 figure quoted elsewhere in this project is a *build-to-build* floor -- three cold starts of one binary give 0.9852 / 0.9842 / 0.9849 because training is not reproducible and the codebooks differ in their last bits. That floor does not apply here. Both arms of this comparison run on one `idx` object with one set of trained codebooks and one assignment, so the difference is paired: nothing varies but alpha. The only residual wobble is the top-ck tie-break, which is order 1e-4.

So most of these deltas are real costs, not noise, and the figure names

```
    the largest rather than calling it the only one. Once the floor is 1e-4,
    openai3-1536 at -0.0028 and bge-m3 across nprobe >= 128 are costs too. The
    honest summary is a range: the rule gives up between 0.0000 and 0.0048 of
    recall, it is largest where nprobe is largest, and the four runs where the
    budget did not change give up exactly nothing.

Over the 32 configurations where the rule did change the budget, payback runs
from 0.5 to 21.0 batches, median 1.75, and the gain is above one in every one
of them (1.044x to 2.653x). The single sub-one gain in the log, 0.996, was one
of the four self-comparisons; removing them removes it.

One caveat this figure cannot remove: the anticorrelation in (a) is largely
definitional. With g the gain, B* = T_cal / (T_fixed - T_rule) =
(T_cal/T_rule) / (g - 1), so B* diverges as g approaches 1 by construction.
What is measured rather than implied is the level -- how large T_cal/T_rule
actually is, and it is small: a median of 1.75 batches means the calibration
costs less than two batches of search even after the gain is taken into
account. Read the panel for the levels, not for the shape.

Both panels read the 36 RULE rows of paper_fronts.log.
```

`fig_negatives` the table-free distance, and reuse as a controlled proxy

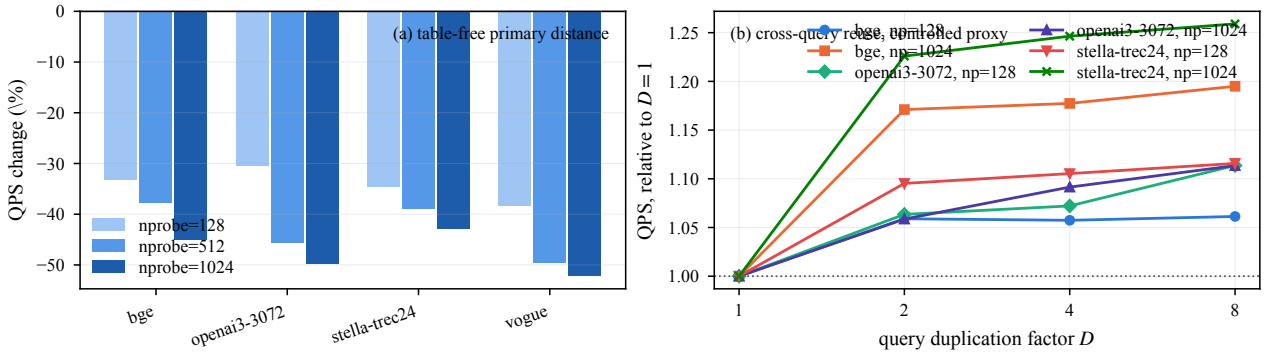


Figure: two negative results with a mechanism, in full.

Section 6.5. The ablation figure gives every negative one bar; these two earn the space because each says something the byte counting does not.

(a) The table-free primary distance. At L=2 the levels are +/−a and D_P = ||q'||^2 + d a^2 − 2a<q',s_y> is an identity -- recall agrees to four decimals in nine of twelve pairs. It cuts per-query shared residency from 4d floats to d, tripling occupancy on openai3-3072, and loses 30–52% anyway, most at high nprobe where the scan dominates. IVF-RaBitQ's paper reports the opposite verdict for the same two kernel shapes on an L40S and says only that it "may vary" with bandwidth; this card has twice the bandwidth, and both sides are measured here.

(b) Cross-query reuse, as a controlled proxy. Duplicating a query D times makes D queries probe exactly the same lists in exactly the same order, which is more agreement than real queries clustered by their coarse assignment ever show. The gain saturates at D=2 -- what a 96 MB L2 already holding the working set looks like -- so the traffic arithmetic overstates what a cluster-centric rewrite has to win from reuse.

It is a proxy, not an upper bound. Duplication holds the schedule fixed and varies only how much the queries have in common; a rewrite would also change the schedule -- amortising the table build across a group, reordering the scan around the list rather than the query -- and those are not on this axis. What this measures is the reuse term alone, and the reuse term is small.

`fig_hierarchy` what the second level buys

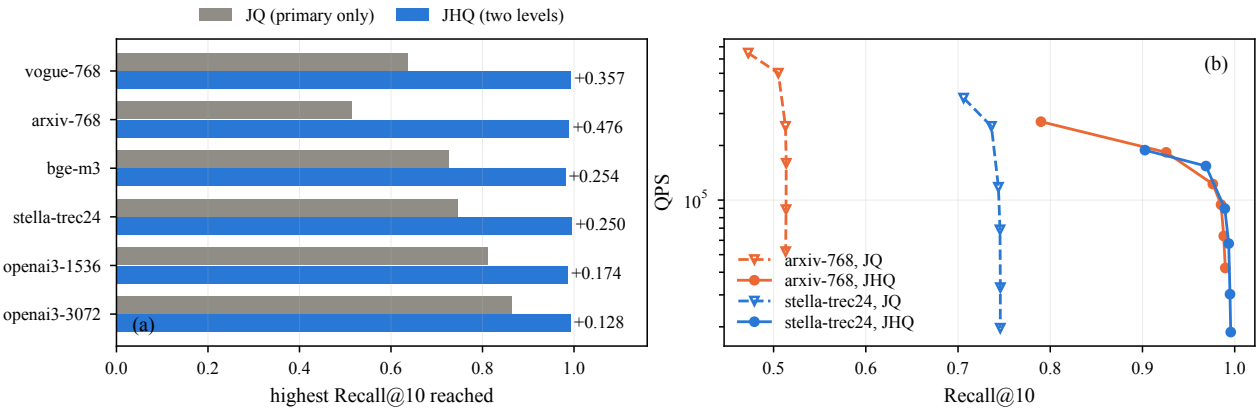


Figure: what the second level buys.

Section 6.7. Everything in the GPU design presupposes that the residual level earns its place -- selective refinement, the alpha budget, the two-level scan -- so the ablation that drops it is the section's premise, not an extra.

Primary codes alone top out between Recall 0.51 and 0.86, and adding nprobe does not fix it: arxiv-768 gains 0.0002 going from nprobe 128 to 1024, because what is missing is resolution, not candidates.

fig_cost JHQ's build, split into training and encoding

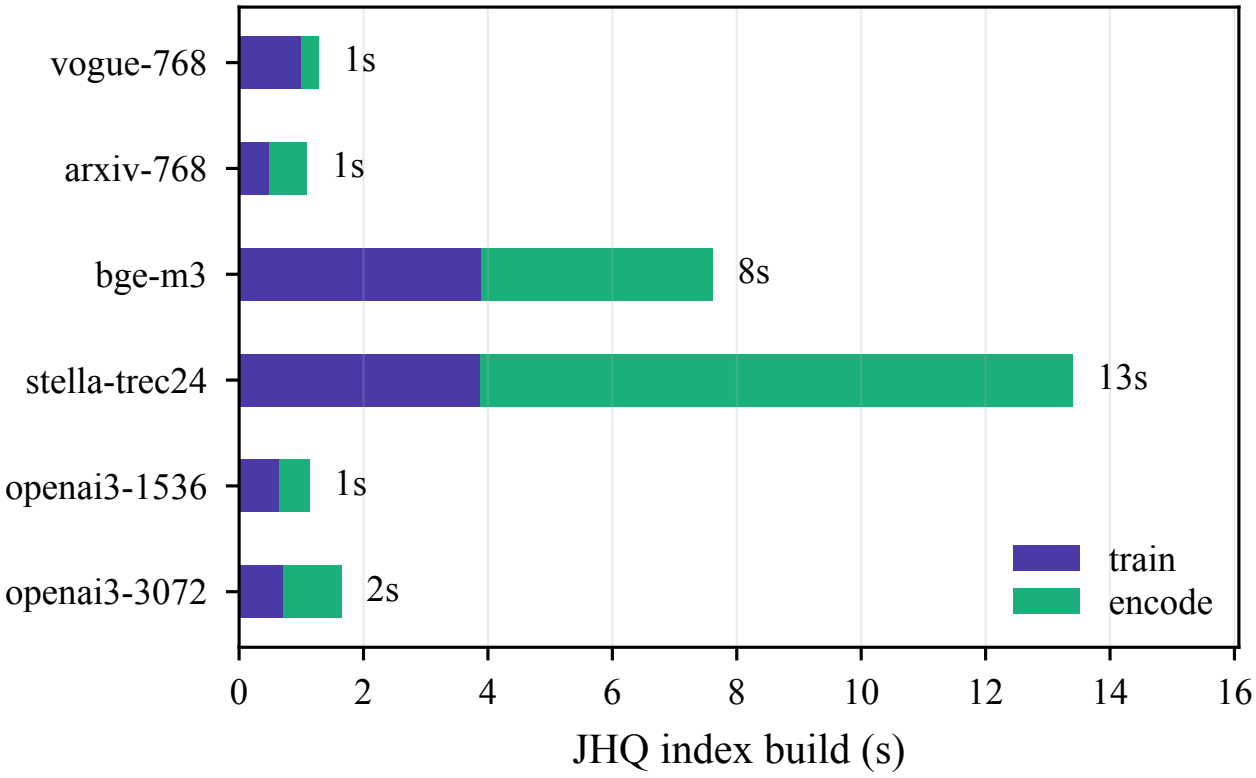
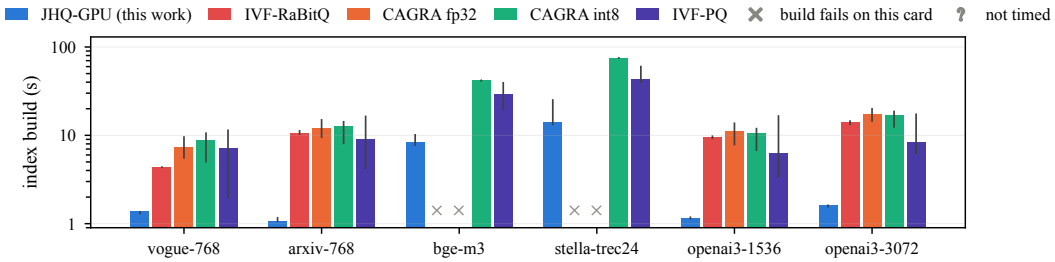


Figure: what the index costs to build and to hold.

Section 6.7. Two panels, both of them numbers that go against this work and are reported anyway.

(a) Resident GPU memory, same call on both sides: cudaMemGetInfo after the index is built and the search workspace allocated. IVF-RaBitQ is smaller wherever it builds, by more than its bits per dimension alone predict -- 8 against 9 is 11%, and the measured gap is 22-39%, because JHQ also carries the selection buffer and the factorised table in its workspace. On the two largest sets IVF-RaBitQ does not build at the parameters its defaults choose; that is a bar that is absent, not a bar at zero.
(b) Build time, JHQ, split into training and encoding.

`fig_build` index build time, all five methods



whisker: the range across swept configurations and repeat builds, not a confidence interval. JHQ's bar is its one cold training plus its median encode: training is cached after a dataset's first row, encoding is not and re-runs every row. All indexes train the coarse quantiser at 39 points a centroid.

Figure: what each index costs to build, on one card.

Section 6.7. Until now this project could say how long JHQ takes to build and nothing about anyone else, which made a section called "Build Time and Memory" half a section: fig_cost's right panel is JHQ's own train+encode split and its x-axis literally reads "JHQ index build (s)".

Every number here was already measured and none of it needed a re-run. The cuVS baselines' build time is the `train_ms` column that `scripts/bench_all.py` has always written; it simply never reached `fronts.json`, which is the file the frontier figure reads. IVF-RaBitQ's is the `build_ms` line of `bench_ivf_rabitq.cu`, and the round-trip through serialize/deserialize is inside it, because without that round-trip the index is not searchable.

What a bar is

Build time is a property of the index, not of the search: rows that share a configuration but sweep nprobe or itopk report the same build. So each bar is the **median across every build of that method on that dataset**, and the whisker spans them.

That spread is not a confidence interval. It runs over two things at once -- the configurations the sweep covered (IVF-PQ on vogue goes from 2.2 s at 96 B a vector to 11.6 s at 768 B) and repeated builds of one configuration (CAGRA int8 at 896 B, ten builds, 4.9 to 6.0 s). A single number per method would have to pick a configuration, and for IVF-PQ there is nothing to pick: its frontier in fig_frontier is a Pareto envelope over several byte sizes, so no one configuration is "the" IVF-PQ on the plot.

Two things that would make this flattering if left unsaid

JHQ's build is a cold build, and the trained-state cache hides it. `paper_fronts.log` reports train=990 ms for vogue at nprobe=8 and train=25.8 ms at nprobe=32 -- the second is a cache hit on the first. Taking a median would report the cache. The bar is the maximum, which is the build that actually happened.

The coarse quantiser is trained at 39 points per centroid, and the build time includes that. `/root/_pa.sh` passes `JHQ_N_TRAIN = 39 x nlist` on every dataset -- 1,277,952 for bge-m3 and stella at nlist=32768, 159,744 for vogue and openai3-3072 at 4096, 319,488 for arxiv and openai3-1536 at 8192. So this is not a cheap build bought by under-training: an earlier draft of this docstring said it was, which was wrong, and the mistake matters in the generous direction -- the bar already carries the cost of the quantiser the frontier was measured on.

A missing bar means one of two different things, and they are drawn apart

The build fails. CAGRA fp32 on bge-m3 and stella (the `oom` field of fronts.json) and IVF-RaBitQ on the same two, where the process aborts with rc=134 even after `train/list` is cut to 32 (`rabitq_bigsets.log`). At 10.1M and 17.8M vectors of 1024 dimensions the allocation does not fit this card, and it is the same reason both are absent from fig_frontier.

The build was never timed. No cell is in this state any more. IVF-RaBitQ on openai3-3072 was in it until the log of `/root/_rqb.sh` was collected into `data/rabitq_o3072_build.log` -- the number had been measured all along and had simply never left the box. The mark stays in the legend because the

```
distinction is worth keeping visible: a gap in our measurements and a
limitation of a baseline are different claims, and drawing them the same way
would assert the second from evidence for the first.
```

`fig_lutgroups` why halves; and the table x layout 2x2

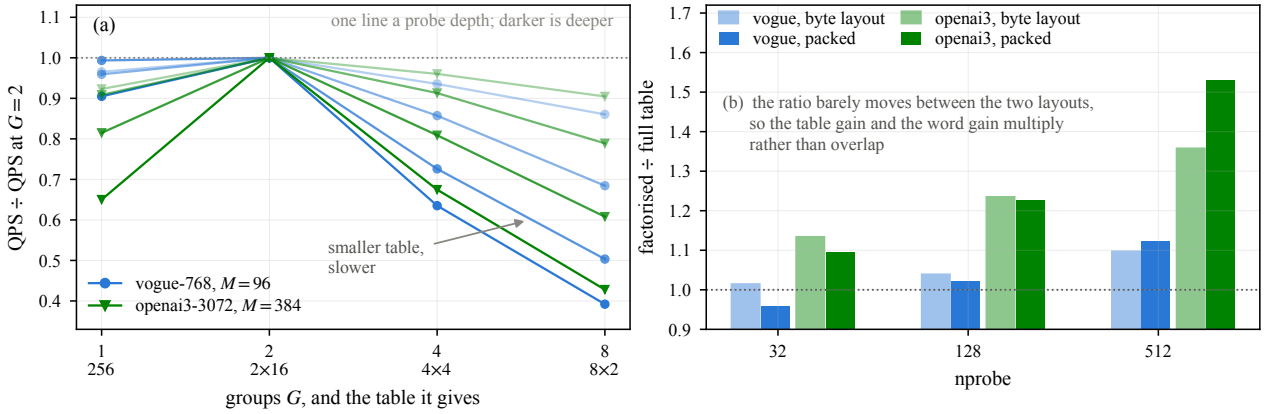


Figure: why the table is split in half, and what the split is worth.

Section 6.4.2. The paper's strongest representation-specific claim is that the primary table factorises exactly, 256 entries to 16+16, because the codebook is a Cartesian product of one-dimensional levels. Two questions follow that the paper could not answer, and both are now measured on one build at one commit (`data/lut_groups.log`, v59).

Why halves, and not quarters?

The identity holds for any partition of the code's eight base-2 digits, so G groups of 8/G bits give G tables of 2^(8/G) entries:

```
G=1  1 x 256 = 256 entries, 1 load a candidate a subspace
G=2  2 x  16 =  32          2                              (shipped)
G=4  4 x   4 =  16          4
G=8  8 x   2 =  16          8
```

Panel (a) is that sweep. All four return identical recall at every point -- they must, the identity is exact -- so the comparison is pure throughput.

G=2 wins all eight configurations, and the losers are informative. G=4 and G=8 have *smaller* tables than G=2 and are monotonically slower, in proportion to their loads a candidate: 2, 4, 8. So the scan is not table-size-bound. Once the table is small enough to sit in shared memory without bank conflicts, shrinking it further buys nothing and costs an instruction per subspace per candidate. That is the same conclusion as the table-free distance in Section 6.5, which moved less memory and still lost, and as the packed load in Section 6.4.3, which moves the same bytes and wins on instruction count. Three experiments, one mechanism: this kernel is issue-bound.

What the factorisation is worth, and when

Panel (b) is the 2x2 the paper had been inferring across version directories: the full table against the factorised one, on the byte layout and on the packed one, all from the same source tree.

The effect is far more M-dependent than `results/v47_split_lut/` reports. That note measured 28 cells at M=96 and M=128 and found +6 to +14.5% with none negative. At M=384 the factorisation is worth **1.23x to 1.53x**; at M=96 it is worth 1.02x to 1.12x, and one cell (nprobe=32, packed layout) comes out at 0.96 -- slightly negative.

The mechanism is visible in the arithmetic. A 256-entry table is M*256*4 bytes: 98 KiB at M=96, which the carveout can still hold, and 393 KiB at M=384, which it cannot, so G=1 there falls back to reading the table from global memory. The factorisation matters exactly where the full table stops fitting. Reported as an average over datasets it is a modest constant; read against M it is a scaling property, which is the more useful claim and the one the paper should make.

The two payoffs are close to independent. On openai3-3072 the factorisation is worth 1.13/1.24/1.36 on the byte layout and 1.10/1.23/1.53 on the packed one -- roughly the same either way -- so the gains multiply rather

```
than overlap. That is the evidence the "pays twice" framing needed and did not
have.
```

`fig_memory` where JHQ\'s memory goes, and why it is above RaBitQ\'s

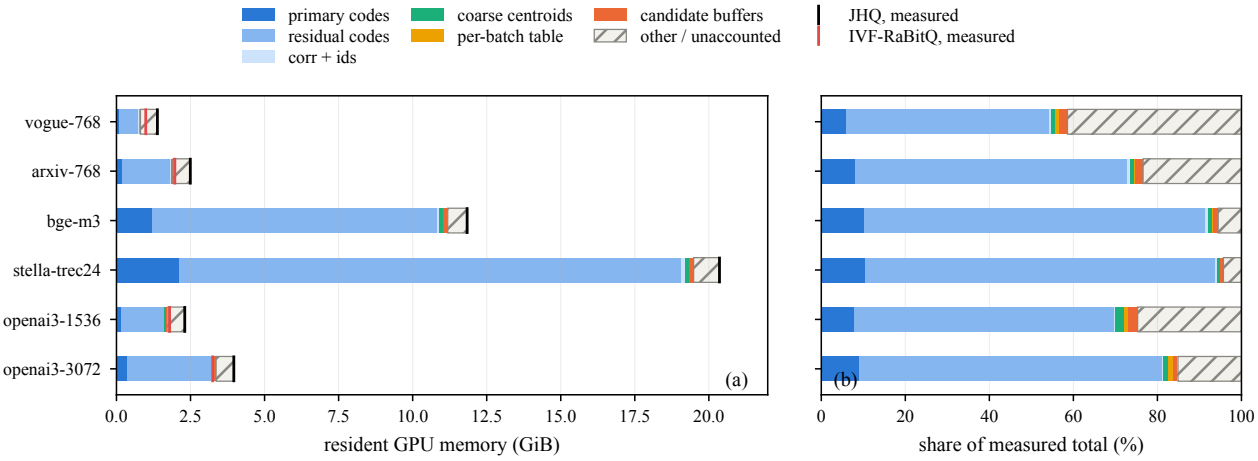


Figure: where JHQ's resident memory goes, and why it is above IVF-RaBitQ's.

Section 6.7, the companion to fig_cost. IVF-RaBitQ is 22-39% smaller wherever it builds, and its bits per dimension alone predict only 11% -- 8 against 9. This is the rest of the gap, and it is not a code-size difference:

* the primary and residual codes are 9 bits a dimension against RaBitQ's 8;
* the coarse centroids are nlist x d floats, the same on both;
* and JHQ carries a per-batch search workspace that IVF-RaBitQ does not need at the same size -- the factorised table for every query in the batch, and the candidate buffers the exact top-alpha-k selection runs in.

The first six bars are computed from the index parameters. They do not add up to what the card actually reports, and the gap is real: the CUDA context, the cuBLAS and cuRAND handles, the allocator's slack, and the training buffers that are freed but whose pool pages are not returned. Rather than let the stack quietly disagree with the measurement, the last segment is *defined* as measured minus modelled, so the bar ends exactly at the cudaMemGetInfo total and the unexplained part is drawn at its true size instead of being left out. It is labelled "other / unaccounted" rather than attributed: the CUDA context and the allocator's slack are what it is *expected* to be, but nothing here measures them separately, and naming a cause we did not measure would be the same mistake as leaving the gap out.